

Research Engineering: 系统级自主 AI 研发模式设计方案

版本: V1.5 评审收口 / 可实现设计基线 (2026-09)

定位: 纯客户端、Git 仓库内、Skills 驱动的 Applied-AI Research Engineering 工作模式

目标关系: 总架构师 (Human) + 高水平技术合伙人 (AI)

研究过程制品语言规则: 由 research-engineering 及其子 Skills 在生成 research artifacts 时执行。给架构师/团队阅读的制品使用中文, 术语和专用词保留英文原词; 供 AI 跨 session 续接、检索或机器处理的制品使用英文。

核心原则: *Make the current uncertainty executable as early as possible.*

有完整系统就跑系统; 没有就跑 vertical slice; 再没有就跑 component; 没有 component 就做 probe / replay / micro-benchmark / architecture spike; 只有当可执行证据暂时不可得时, 才停留在纯推理、论文和源码研究。

内容导航

1. 设计结论与核心思想
2. 为什么不能沿用软件工程型 Agent Workflow
3. 最终概念模型：Evidence 是稳定抽象
4. 人机分工：Architect-in-the-loop
5. Architect Signals：资深工程师如何及时纠偏而不接管研究
6. Research Subject、Evidence Ladder、Research Environment 与多级 Promotion
7. Fast Loop、Slow Loop 与 Research / Integration 双模式
8. Minimum Correctness Envelope：替代重型 TDD
9. 跨 Codex / Claude Code 的 Skills 架构
10. research-engineering 主 Skill 设计
11. 核心入口 Skills 与专业专家 Skills
12. 轻量持久状态与 Session Continuity：ACTIVE / CURRENT / ARCHITECT / BOUNDARIES / ENVIRONMENT / FINDINGS / EVIDENCE
13. 可选本地工具：researchlog.py，不是 orchestrator
14. Git / GitHub Research Infrastructure、worktree、evaluator isolation 与可信研究
15. 与 Superpowers / GSD 的共存策略
16. Codex 中的日常实战 workflow
17. 具身安全比赛：从 Day 0 到完整系统的贯穿示例
18. RAG / Agent / VLM / 多模态系统的泛化方式
19. 开源实现复用矩阵
20. V0 / V1 / V2 分阶段落地路线
21. 如何评价这套方法是否真正更高效

22. 风险与反模式
23. 最终建议
24. 附录：可直接复制的核心文件草案
25. 研究过程制品语言规范（由 Skills 执行，Normative）
26. 原生客户端可执行性验收（Normative）
27. 参考资料

1. 设计结论与核心思想

本方案的目标不是再造一个 GSD，也不是在 Codex / Claude Code 外面套一个庞大的 autonomous-agent server。它是一层很薄的 **Research Operating Layer**：通过少量 Skills、Git、研究状态文件和可执行证据，让通用 Coding Agent 的默认行为从“完成软件需求”切换为“持续进行系统级技术研究”。

最终角色模型是：

- **你是总架构师 / Research Lead。** 负责战略方向、系统级判断、观察物理/仿真效果、定义硬边界、及时纠偏、重大架构裁决以及最终 promotion。
- **AI 是技术合伙人 / Technical Research Partner。** 在给定方向和边界内，自主完成复杂算法选型、底层模型和系统机制实现、快速 prototype、运行验证、日志/trajectory 分析、竞争假说构造、下一实验选择和连续迭代。
- **真实运行结果是裁判。** 研究阶段首先追求 validated technical progress / information gain，而不是代码覆盖率、计划完整度或单元测试数量。



1.1 十八条设计不变量

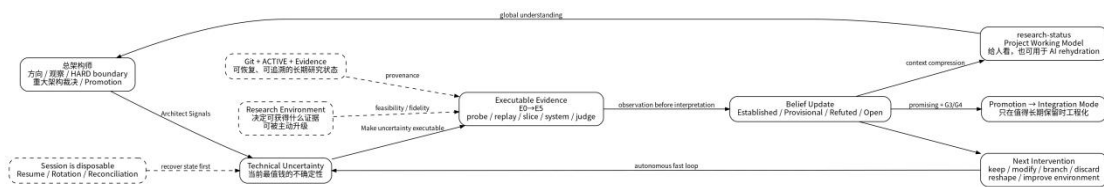
1. **Evidence 是贯穿全生命周期的稳定抽象。** 不假设项目开始时已经存在 runnable candidate 或 evaluator。
2. **不确定性优先变成可执行问题。** 能跑就不要只讨论；能做 probe 就不要先实现完整系统。

3. **研究对象可以是任何粒度。** Idea、算法、机制、component、architecture spike、research harness、partial stack、vertical slice、完整系统都可以成为 Research Subject。
4. **AI 对 routine 技术决策默认自治。** 不把算法、阈值、库、模块内部实现选择反复抛给人。
5. **架构师可以随时纠偏，但干预是 steering impulse，不是 takeover。** 通过明确的 Architect Signal 语义改变 AI 的 research policy / search space。
6. **Research Mode 不采用教条式 TDD。** 只保留保护实验可信性的 Minimum Correctness Envelope。
7. **原始证据 append-only；信念和 CURRENT 可重建。** 聊天上下文不是 source of truth。
8. **Session 是可丢弃的执行上下文。** 任何 decision-relevant state 都必须落在 repo / Git / research artifacts 中；新 session 不依赖旧聊天即可恢复。
9. **复杂失败不允许一眼生成单一 root-cause story。** 先重建 trace，再维护 competing hypotheses，允许 INCONCLUSIVE。
10. **好的研究结果可以是失败。** INFRA_FAILED、ENV_BLOCKED、EVIDENCE_INVALID、informative_failure、refuted 等必须区分。
11. **工程化发生在 promotion 之后。** Superpowers/GSD/TDD 的强工程纪律用于被证明值得保留的机制，而不是每次 mutation。
12. **Research Environment 是一等研究对象。** “当前环境做不了”不能被解释为 hypothesis 失败；AI 应先 reshape/decompose experiment、构造 valid surrogate/harness，必要时才投资升级环境。
13. **Git 是 normative core research infrastructure；GitHub 是 optional remote research plane。** Git 承担 executable state、provenance、branching、checkpoint 与 promotion；GitHub 可承担远端 durability、独立 verifier 和协作，但不能成为 canonical state 的唯一来源。
14. **Skills 必须按 artifact audience 选择语言。** 研究过程中生成的 Human-facing artifacts 默认使用中文并保留英文术语；AI-facing durable state、instructions 与 machine-readable records 使用英文。该规则由 research-engineering 执行，不要求把所有项目文件机械地统一成某一种语言。
15. **总架构师必须能随时获得全局 Project Working Model。** research-status 将分散在 ACTIVE/CURRENT/ARCHITECT/BOUNDARIES/ENVIRONMENT/FINDINGS/EVIDENCE/Git 中的状态压缩为中文、证据可追溯、同时对 AI 上下文重建有效的 derived snapshot；它不成为第二份 canonical state。

16. **Day 0 必须有显式 Bootstrap，而不是让主循环凭空假设状态已存在。** research-bootstrap 负责把普通 repo / 赛题资料 / SDK / baseline 初始化为最小 Research Engineering 状态，不生成重型项目计划。
17. **Evaluation Surface 也是 Research Subject。** 没有 evaluator、metric 与真实行为不一致、official judge 昂贵或 local proxy 失真时，必须允许 AI 研究并改进评价能力，而不是在坏 metric 上盲目优化。
18. **机器状态必须可演化且 crash-safe。** Canonical machine-readable artifacts 使用 schema_version；未知新 schema 不得静默覆盖；关键状态采用 validate + atomic replace，长时实验和 mutable inputs 必须有可恢复 identity。

1.2 全局设计思想：一个闭环、三个平面、三种成熟度、两种模式

这套方案不是“更多项目管理流程”，而是把 Coding Agent 改造成一个**可持续研究的闭环控制系统**。最核心的循环只有一个：



Technical Uncertainty

→ cheapest valid executable evidence

→ Observation

→ Belief Update

→ Next Intervention



总架构师不成为每轮 scheduler，而通过 Architect Signals 对这个闭环施加高信息量 steering；AI 在边界内负责 routine 技术判断和连续实验。research-status 把闭环当前状态压缩为总架构师可读、也能被 AI 重新吸收的 Project Working Model。

整个系统可以理解为三个互相正交的平面：

平面	解决的问题	核心机制
Research Control Plane	下一步研究什么、谁有决策权	research-bootstrap / research-engineering / expert

平面	解决的问题	核心机制
		Skills / Architect Signals
Durable	session 丢失后如何继	ACTIVE / CURRENT / ARCHITECT / BOUNDARIES /
Knowledge Plane	续、已经知道什么	ENVIRONMENT / FINDINGS / Evidence
Execution & Trust	真实运行状态、可追溯	Git / run manifest / artifacts / evaluation surface /
Plane	性、评价可信度	environment / optional GitHub verifier

同时必须独立追踪三种 maturity:

System Maturity 系统实际上长到了哪里
Evidence Maturity 对这些能力知道得有多可靠
Environment Maturity 当前实验环境能验证到哪里

它们不能压缩成一个“完成百分比”。系统可能已经 E4 可运行，但一个新机制仍只有 E1 Evidence；也可能系统实现完整，但当前 environment 对某类 attack 只能验证到 E2。

最后，整个生命周期只有两种主要工作模式：

Research Mode

高自治 / 快速 probe / disposable code / 最小 correctness
↓ evidence supports promotion

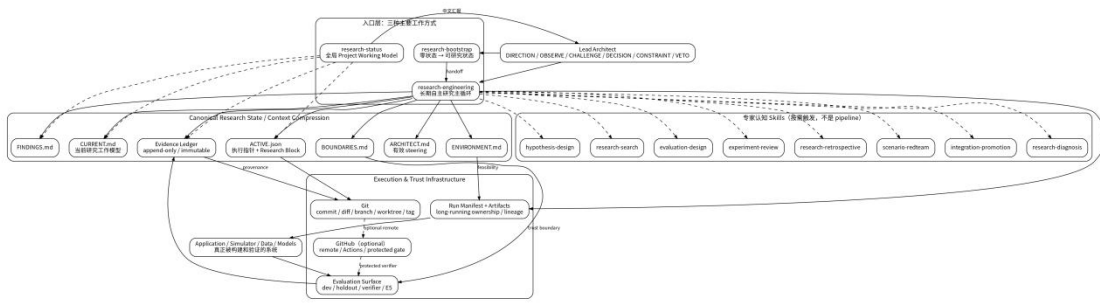
Integration Mode

stable interface / selective TDD / regression / clean reproduction
↓ promoted baseline

Research Mode

这就是整套设计的控制逻辑：**Research Mode** 发现什么值得存在，**Integration Mode** 负责让它可靠地长期存在。

1.3 总体设计架构：从总架构师到真实系统的完整栈



从上到下，各层职责是：

1. **Lead Architect Interface**: 人提供 Direction、Observation、Challenge、Decision、Constraint、Veto；通过 research-status 获取全局状态。
2. **Top-level Entry Skills**: research-bootstrap 负责零状态初始化，research-engineering 负责长期自主循环，research-status 负责只读全局认知。
3. **Expert Skills**: diagnosis、hypothesis、search、evaluation、review、retrospective、red-team、promotion 按认知任务触发，绝不组成固定 pipeline。
4. **Canonical Research State**: 保存 decision-relevant knowledge 和 execution pointer；任何 session 都可以丢弃。
5. **Execution & Trust Infrastructure**: Git 给出精确 executable state，run manifest 管理真实运行，Research Environment 决定证据可行性，Evaluation Surface 决定证据是否能回答目标问题。
6. **Application / Simulator / Data / Models**: 真正被构建、攻击、测量、验证和最终 promotion 的对象。

三条 source-of-truth 规则贯穿全栈：

What are we doing now? → ACTIVE + Git + run manifest

What have we learned? → Evidence → FINDINGS / CURRENT

What must we obey? → ARCHITECT + BOUNDARIES + ENVIRONMENT

STATUS.md、GitHub Issue/PR、中文 Research Brief 都只是 derived / coordination views，不得反过来覆盖 canonical state。GitHub、hooks、subagents、MCP、daemon 都可以增强体验，但删除它们后核心研究语义必须仍成立。

典型生命周期因此是：

ordinary repo / problem assets

↓ research-bootstrap

minimal durable research state

↓ research-engineering

E0 → E1 → E2 → E3 → E4 / E5 evidence

↑ research-status / Architect Signals

↑ environment & evaluation improvement

↓ promising mechanism

Integration Mode / promotion

↓ stable baseline

return to Research Mode

1.4 明确非目标

V0/V1 **不做**：

- 通用多 Agent Server；
- 工作流 DAG / 队列 / daemon；
- MLflow 替代品；
- 全项目自动 CI 平台；
- 每次变更自动完整回归；
- “计划 → 审核 → TDD → Review → 下一步”的研究流水线；
- 必须存在标量 metric 的自动优化器；
- 一开始就完整实现 solution frontier / Monte Carlo tree / population search。

这些能力后面可能出现，但必须由真实项目痛点驱动，而不是因为“一个 autonomous framework 看起来应该有”。

2. 为什么不能沿用软件工程型 Agent Workflow

Superpowers、GSD 等软件工程方法本身并不差；问题在于其优化目标不同。Superpowers 的 systematic-debugging 在证据优先、边界 instrumentation、backward tracing 等方面非常强，但整体 Superpowers 体系会自动引导到 brainstorming、plan、TDD、verification、full regression 等可靠软件交付纪律。[S4][S5]

传统软件工程隐含的是：

需求基本已知

- 设计逐渐稳定
- 计划分解
- 正确实现
- 验证完成

而 Applied-AI Research 的真实状态是：

目标 / 约束已知

但“应该造什么”和“为什么会有效”未知

- 架构本身是变量
- 模块不断被丢弃/替换
- 大量失败是必要信息
- evaluator 甚至也是逐步建立的

因此 Research Engineering 不是“轻一点的 TDD”，而是**另一种控制论**：

Uncertainty

- executable evidence
- observation
- belief update
- next intervention

软件工程问：“**我们是否正确实现了预定设计？**”

Research Engineering 问：“**刚才真实系统教会了我们什么？下一条最值钱的证据是什么？**”

3. 最终概念模型：Evidence 是稳定抽象

早期 Deep Research 的一个关键修正：不能把 Candidate + Evaluator + Run 当成系统起点。对复杂 AI 应用，一开始往往只有：赛题/业务说明、数据、SDK、仿真接口、已有 foundation model、baseline 片段和几个模糊的大方向。

因此顶层统一模型应是：

Research Question / Technical Uncertainty



Research Subject

idea / mechanism / component /

architecture / slice / system



Evidence Acquisition

reasoning / probe / replay / spike /

partial loop / simulator / judge



Observation



Belief Update



Modify / Assemble / Discard / Branch



这里 candidate system 和 evaluator 只是项目成熟后自然出现的特殊形式。

还要同时追踪第三个独立维度：**Research Environment Maturity**。系统做到了什么（System Maturity）、我们对它知道得有多可靠（Evidence Maturity）、当前研发环境能验证什么（Research Environment Maturity）三者不能混成一个“项目完成度”。很多时候下一个最高价值动作不是修改 candidate，而是先制造更好的实验仪器：replay converter、attack injector、trace hook、simulator adapter、mock/physics surrogate 或 local holdout。

3.1 Construction Research 与 Optimization Research

同一个协议覆盖两个阶段：

Construction / System Formation Research:

- 从需求和资产理解问题；
- 识别 technical unknowns；
- 做 feasibility probe；
- 做 architecture spike；
- 比较算法机制；
- 建立一个组件；
- 为组件临时构建 replay/microbench；
- 把多个模块连接成 partial stack / vertical slice；
- 第一时间触碰真实 simulator / hardware proxy。

System Optimization / Failure-driven Research:

- 完整场景下出现 failure；
- trace reconstruction；
- failure taxonomy；
- competing hypotheses；
- discriminating experiments；
- robustness / red-team；
- rubric vector、latency、safety、recovery 等多目标权衡；
- 外部 judge 校准。

它们共享同一 Evidence Ledger；不是两套 workflow。

4. 人机分工: Architect-in-the-loop

Human-in-the-loop 容易被理解成“每一步需要人审批”。这里更准确的是 **Architect-in-the-loop**：AI 连续自主工作，人只在高价值处施加架构级信号。

4.1 总架构师负责什么

总架构师拥有最终权力，但不承担日常 research scheduler：

- 明确产品/比赛目标与战略方向；
- 判断系统是不是“整体走偏”；
- 从真实/仿真表现中提供 AI 难以获得的 gestalt observation；
- 维护 HARD 边界；
- 对重大架构取舍做裁决；
- 决定官方提交、额外计算预算、外部数据、付费资源；
- 决定何时把已验证机制 promotion 成长期稳定基线。

4.2 AI 技术合伙人负责什么

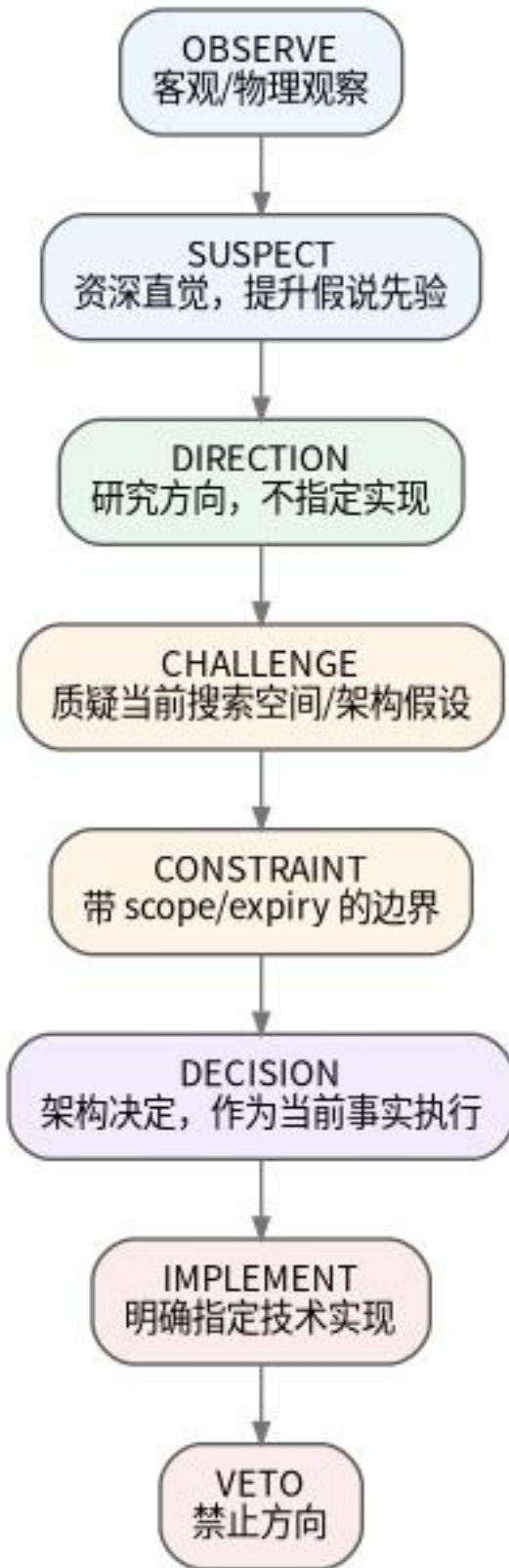
AI 默认拥有 routine research autonomy：

- 阅读赛题、代码、SDK、数据；
- 选择需要深入搜索的算法/论文/开源实现；
- 识别当前最大技术不确定性；
- 自己生成并淘汰候选算法；
- 构造最便宜的 executable research surface；
- 快速实现 prototype / instrumentation；
- 运行 probe/replay/simulation；
- 读取真实 trajectory/log/metrics；
- 形成 competing hypotheses；
- 设计判别实验；
- 决定 keep / modify / branch / discard；
- 在 plateau 时主动改变技术路线；
- 更新持久研究状态。

它不应该因为“不知道下一步用 Kalman 还是 Bayesian / learned model”就问架构师。技术不确定性应该触发 research，而不是 escalation。

5. Architect Signals: 资深工程师如何及时纠偏而不接管研究

这是扩展讨论中最重要的新机制。资深架构师的一句话可能是观察、怀疑、方向，也可能是必须执行的决定；如果 AI 无法区分，就会把你的“随口一猜”机械理解成实现命令。



5.1 八类 Architect Signal

Signal	含义	AI 默认响应	是否可挑战
OBSERVE	物理/仿真/代码现象	作为 evidence；主动量化/定位	不需要挑战，需验证其机制解释
SUSPECT	资深技术直觉	提升相关假说先验，优先便宜验证	是
DIRECTION	研究方向/优先级	自主选择算法和实验推进	是，可带证据 push back
CHALLENGE	认为当前研究路径可能错	暂停当前窄 exploitation，重新诊断 /retrospective	不应忽略
CONSTRAINT	当前边界	加入有效边界，受 scope/expiry 限制	原则上否，除非请求修改
DECISION	架构师已作出的架构决定	记录并执行；可一次性指出技术风险	否；除非被后续 Decision 取代
IMPLEMENT	明确要求具体实现	执行，同时负责验证	可一次性警告明显风险
VETO	禁止某方向	立即停止该方向并记录原因	否

正常研发应尽量停留在表格上半部。你的经验主要用于**异常感知、搜索空间剪枝、架构 invariant**；算法内部细节尽量留给 AI。

5.2 每条 Signal 建议具有三个属性

strength: DIRECTION
scope: next_research_block
expiry: after_6_evidence_iterations

关键是 scope + expiry。否则历史上的临时意见会逐渐变成不可挑战的架构教条。

例：

type: CONSTRAINT
scope: current_recovery_research
expiry: recovery_checkpoint
statement: Do not modify navigation planner.

它不能被写成永久 NEVER MODIFY NAVIGATION。

5.3 Challenge Protocol: AI 不是机械服从

OBSERVE / SUSPECT / DIRECTION / IMPLEMENT 默认允许 AI 做**一次技术性 push back**：

“当前 trace 表明问题更像离散状态释放，而不是连续 measurement noise。直接实现 Kalman 不能区分关键假说。我建议先跑一个 12-case replay；成本很低。如果结果仍支持 filtering，再实现 Kalman。”

这才是“技术合伙人”。

但架构师可用：

DECISION FINAL：

按两级 safety boundary 实现，不再花时间验证是否需要该边界。

这时 AI 应停止争论。

5.4 Autonomy Return

人一旦纠偏，AI 不应继续等待后续十步指令。

协议规定：

Architect intervention

→ ingest / validate / update policy

→ execute immediate correction

→ automatically return to autonomous Research Mode

Architect intervention 是 impulse，不是 takeover。

5.5 最不推荐的干预：半实现级建议

不推荐：

“把最近 5 帧存缓存，做平均，低于 0.6 就进入 recovery……”

这已经替 AI 完成了问题理解、算法选择、representation、阈值和实现。

更好：

SUSPECT:

当前 safety confidence 太依赖瞬时 observation，我怀疑缺少 temporal memory。
自行研究最合适的机制；延迟不能明显增加。

6. Research Subject、Evidence Ladder、Research Environment 与多级 Promotion

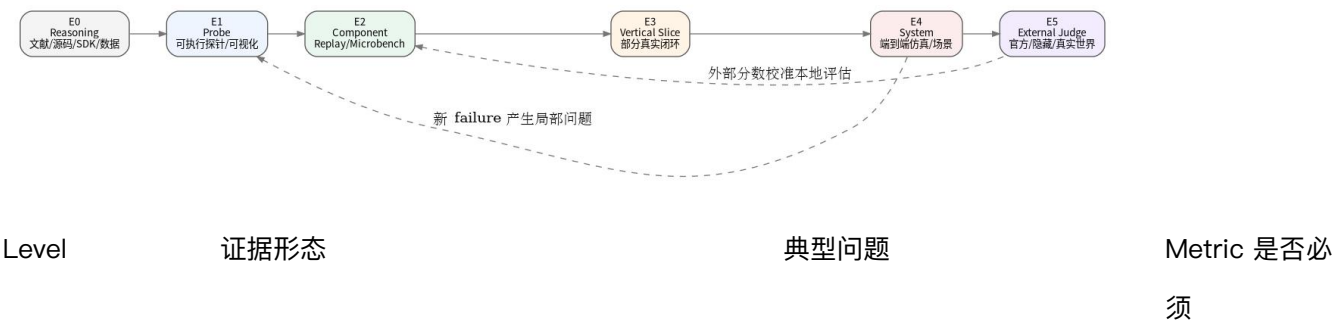
6.1 Research Subject

统一用 research_subject 代替“parent runnable candidate”。推荐类型：

question
idea
mechanism
algorithm
component
architecture_spike
harness
instrumentation
evaluation_surface
partial_stack
vertical_slice
system
submission_candidate

这样第一天只有一个 NumPy probe 和三周后完整机器人系统都能被同一协议描述。

6.2 Evidence Ladder



Level	证据形态	典型问题	Metric 是否必须
E0 Reasoning	文献、源码、SDK、数据探索、架构推演	值不值得试？信息/接口是否存在？	否
E1 Probe	一次性脚本、可视化、synthetic probe、feasibility spike	机制是否产生预期信号？	否，观察可以成立
E2 Component	replay、microbenchmark、孤立模块比较	A/B 哪个机制更好？	通常有
E3 Vertical Slice	若干真实组件 + 明确 mock 的部分闭环	信息/动作能否沿关键因果路径正确传播？	通常有
E4 System	端到端应用/仿真/场景 suite	系统在真实任务分布上行为如何？	是，多维
E5 External	官方 judge、隐藏集、真实世界	本地结论是否泛化？	外部给定

这些 level 是**证据成熟度，不是软件完成度**。成熟系统仍然经常为了一个新机制回到 E1。

6.3 最便宜的高信息证据原则

AI 每轮先问：

What is the cheapest executable artifact that can materially reduce the current technical uncertainty?

不是“什么代码最容易写”，而是“哪种 evidence / cost 比最高”。

例：

- 如果只想知道视觉与 LiDAR disagreement 是否有信号，没必要先接 VLA、planner 和 controller；
- 如果想判断 safety governor 放在 planner 内还是 planner 后，做两个 200 行 architecture spike 比写 10 页设计文档更有价值；
- 如果一个 recovery failure 只能在闭环时出现，component microbench 再漂亮也不能替代 E3/E4。

6.4 Promotion 是多级的



Idea

- Probe
- Component
- Partial stack / Subsystem
- Vertical Slice
- System Candidate
- Promoted Baseline

每一级只做**进入下一阶段所必要的最小工程化**。局部 probe 有效，不意味着马上进入完整 TDD 和全量 regression。

6.5 Research Environment Envelope: 研发环境决定可获得的证据

Research Engineering 不能把“实验能否执行”看成纯基础设施问题。当前研发环境直接决定了**哪些 research question 可以被什么强度、什么可信度的 evidence 回答**。因此 Research Environment 与 System Maturity、Evidence Maturity 是三个独立维度：系统做到了什么、我们知道得多可靠、我们现在有能力验证什么，三者不能混成一个“项目完成度”。

研发环境至少包括：compute / device、OS/runtime、model serving、simulator/SDK、replay/data、external service、research harness、instrumentation、evaluation surface，以及这些能力的已知限制和替代路径。

6.5.1 实验不可行状态必须严格分类

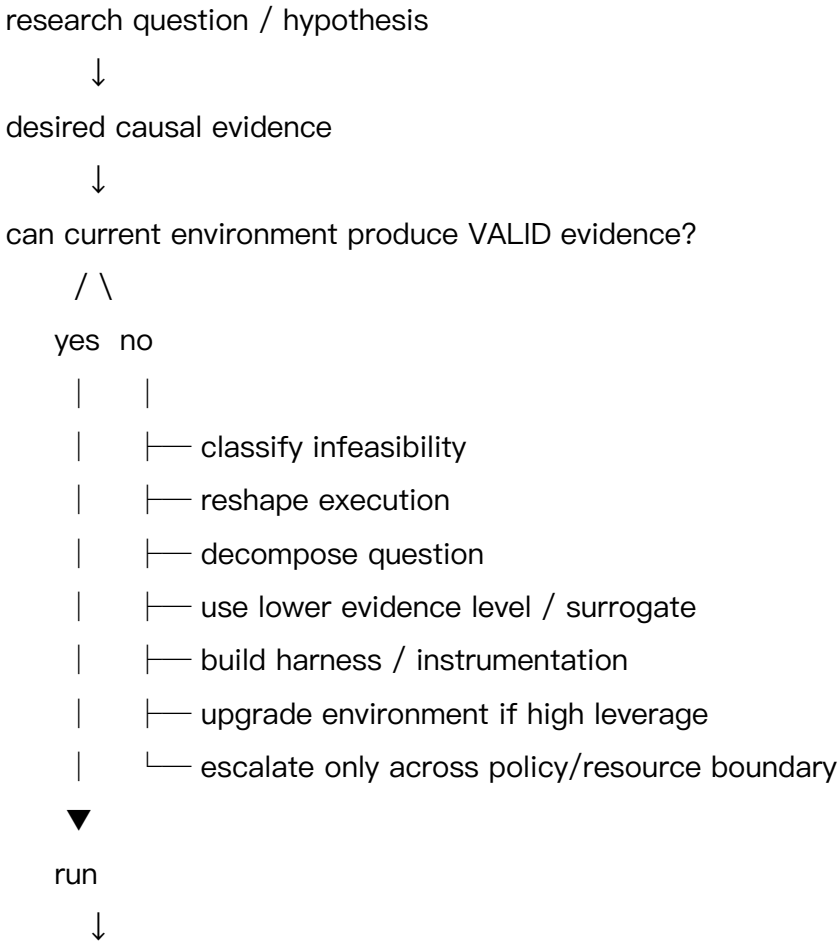
状态	精确定义	是否能否定 hypothesis
INFRA_FAILED	本次 run 按当前环境本应可执行，但因 crash、script bug、临时服务故障等没有产生有效 evidence	否
ENV_BLOCKED	环境理论支持，但当前缺 dependency、配置、权限、service、credential 等前置条件	否
ENV_UNSUPPORTED	当前 simulator/SDK/research surface 从能力上不表达研究所需现象	否

状态	精确定义	是否能否定 hypothesis
RESOURCE_EXCEEDED	当前实验设计超过显存、CPU、storage、wall-clock 或预算 envelope	否
EVIDENCE_INVALID	替代实验可以运行，但没有保留关键 causal feature，因此不能回答原 research question	否
SCIENTIFIC_NEGATIVE	环境有效、实验有效、evidence 能回答问题，但机制确实未表现出预期效果	是

这些状态必须与 research_outcome 分离。OOM、SDK 不支持或 context/session interruption 都不能进入 FINDINGS.md 形成“算法无效”的 durable belief。

6.5.2 Environment-constrained research state machine

实验不可行时不停止研究，也不立即询问架构师：



bound conclusion to actual evidence level



resume research

核心原则: **seek the closest valid evidence, not merely the easiest runnable substitute.**

6.5.3 Surrogate Validity Contract

任何 surrogate / mock / replay / reduced simulator 在被用于支持 research conclusion 前, AI 必须在内部明确以下六项; 可写入 Evidence record, 不要求生成单独文档:

target_causal_claim: <what is actually being tested>

required_causal_features:

- <feature that must exist for the claim to be meaningful>

preserved_features:

- <what the surrogate preserves>

missing_or_distorted_features:

- <what is absent or simplified>

allowed_conclusions:

- <what this evidence may support>

forbidden_conclusions:

- <what must not be inferred>

verdict: VALID_SURROGATE | EVIDENCE_INVALID

例如使用 recorded trust trajectory + 原 recovery state machine 可以验证“离散 recovery release 是否过慢”, 但不能证明“闭环机器人不存在物理 oscillation”; 没有 dynamics 的 mock 不能用于证明摩擦变化下的动力学稳定性。

6.5.4 Environment comparability contract

环境变化可能制造假 improvement。需要记录 compact fingerprint, 但不能因为任何 hash 变化就触发 full regression。

潜在 material changes 包括:

- simulator / physics engine version、physics timestep / tick rate;
- sensor model/noise/randomization distribution;
- dataset/replay snapshot;

- model checkpoint / external model/API version;
- inference backend、quantization、compiler/runtime mode;
- evaluator implementation / metric semantics;
- hardware (当 latency/resource 指标参与结论时)。

comparison 状态建议只有三种:

COMPATIBLE direct comparison allowed

REBASELINED comparison allowed after rerunning anchor baselines

INCOMPARABLE do not attribute cross-environment delta to candidate mechanism

发生 material change 时优先重跑少量 anchor baseline，而不是全量回归。

6.6 Environment investment: 研究系统和被研究系统共同演化

Research Environment 本身可以成为 Research Subject。harness、instrumentation、evaluation_surface 不是“杂活”，而是能提高未来 evidence throughput 的研发资产。

环境能力不足时，AI 默认按以下顺序自主处理：

1. **Reshape**: 不改变 causal question，仅改变 evidence acquisition，例如 full sim → recorded replay。
2. **Decompose**: 把大问题拆成当前环境可以可靠回答的子问题。
3. **Valid Surrogate**: 降低 Evidence Level 但明确 Surrogate Validity Contract 和 conclusion boundary。
4. **Upgrade Environment**: 若薄 harness 能解锁大量后续研究，主动实现 attack injector、replay converter、trace hook、sim adapter、microbenchmark 等。
5. **Escalate**: 只有需要新硬件、license、付费 API、官方权限、物理机器人、重大 simulator 改造、大量资源投入或 HARD boundary 变化时找架构师。

AI 可以自主建设 FREE/PROVISIONAL 范围内的低成本研究基础设施，不应为 replay converter、trace instrumentation、小型 attack injector、local adapter、disposable mock 逐项请求批准。

环境投资采用轻量判断：

Environment investment value

$\approx \text{future research unlocked} \times \text{expected reuse} \div \text{implementation cost}$

不要求形式化打分。一个 30 分钟 converter 若能支持未来 20 个实验应优先；为一个低概率 idea 花两天重写 simulator 通常不值得。

6.6.1 ENVIRONMENT.md 生命周期

只有以下事件才需要更新 ENVIRONMENT.md:

1. 新发现一个 previously unknown capability / limitation;
2. 新增 reusable harness/adapter, 改变可获得的 Evidence Level;
3. material environment change 影响 comparability;
4. capability 从 blocked/unsupported 变为 available, 或反向变化。

每项重要 capability/limitation 应尽量引用 verified_by: EV-*, 避免几周后无法知道结论来源。

7. Fast Loop、Slow Loop 与 Research / Integration 双模式

7.1 Fast Loop

默认连续研究循环:

current uncertainty

- competing explanation / mechanism
- choose discriminating evidence
- minimal implementation / instrumentation
- Minimum Correctness Envelope
- RUN
- inspect actual output
- append evidence
- keep / modify / branch / discard
- next uncertainty

强调:

- routine 轮次不写长方案;
- routine 轮次不请求用户 approval;
- routine 轮次不跑 full regression;

- 如果运行能回答，就优先运行；
- 实验单位是 **evidence-producing iteration**，不是 commit 数或 task 数。

7.2 Slow Loop / Research Retrospective

触发条件可以是：

- 约 8—15 个有效实验；
- 5+ 次相似变化无实质进展；
- local metric 与物理/仿真观察开始冲突；
- 当前 architecture assumption 被新 evidence 动摇；
- Agent 连续在同一参数/机制族内做 exploitation；
- 新的 dominant failure 已经转移。

Slow Loop 问的是：

我们真正知道了什么？

哪些 belief 其实证据不足？

哪个临时 architecture assumption 已变成 dogma？

研究预算是否集中在错误的模块？

哪个失败质量最大？

应该继续 exploit、探索新机制，还是先理解 failure？

它更新 FINDINGS.md 与 CURRENT.md，不改 raw evidence。

7.3 Research Mode vs Integration Mode

维度	Research Mode	Integration Mode
优化目标	信息增益 + validated behavior / 时间	稳定、可复现、长期可维护基线
代码	可丢弃、允许局部脏	应维护
Unit tests	只写关键边界/复现断言	为已证明机制补选择性 regression
Refactor	能不做就不做	应做
Backward compatibility	PROVISIONAL 接口不保证	稳定 contract 要保证

维度	Research Mode	Integration Mode
Full regression	默认不跑	Promotion 必跑
Instrumentation	大量允许	删除或正式化
人工 review	方向性 / checkpoint	架构师 promotion review
AI autonomy	很高	中等，需保护已晋升行为

Superpowers/GSD 的主要位置是 **Integration Mode**，不是 Fast Loop。

8. Minimum Correctness Envelope: 替代重型 TDD

TDD 在生产工程中很有价值，但在大量一次性 hypothesis mutation 中，如果每次都 RED-GREEN-REFACTOR、补单元测试、长计划、全回归，会让 research throughput 崩溃。这里不取消测试，而是改变测试的目的：

测试只保护实验可信性和被证明值得长期保留的行为。

8.1 五类 Gate

Gate	核心问题	例子	何时
G0 Executability	这次能产出 evidence 吗？	syntax/import/startup/不立即 crash/预期 artifact 出现	每次 empirical run
G1 Boundary Integrity	基础 contract 错误会不会使结论无效？	shape/schema/坐标系/单位 /timestamp/control range/API contract	相关时必跑，少量断言
G2 Research Validity	这次 evidence 真能回答问题吗？	baseline/control、数据版本、无明显 confound、provenance	宣称结论之前
G3 Candidate Robustness	是否超出 motivating case 仍有价值？	clean guard、代表性 slice、少量重复、latency/cost	promising 才跑
G4 Integration Promotion	是否值得变成稳定平台？	clean checkout 重现、更广回归、关键 regression、ablation、judge integrity	promotion / submission

8.2 Gate 随 Evidence Level 演进

E0: 确认读的是正确规则/SDK/源码；区分事实与推测。

E1: 脚本能执行；先查看一份 raw input；检查 shape/单位/坐标/时间；明确 probe 的限制。

E2: 固定 replay/config；保留 control；接口和输出 semantics 可复现。

E3: 明确哪些模块是真实、哪些是 mock；记录跨模块 boundary trace 和 latency。

E4: 代表性 scenarios、clean case、safety/rubric vector、必要的重复/seed。

E5: judge 隔离、submission budget、禁止根据 hidden truth 调 candidate。

8.3 哪些测试值得长期保留

研究阶段仅优先保留四类测试：

1. **Hard boundary assertion**: 坐标、单位、schema、安全动作范围等。
2. **Experiment-validity assertion**: 数据泄漏、错误 split、judge truth 泄漏等。
3. **高价值 bug reproduction**: 一个关键 bug 曾多次复发，极小测试可廉价锁住。
4. **Promoted mechanism regression**: 只在 Integration Mode 中建立。

不默认给 transient helper、临时 parser、probe 中每个函数写 unit tests。

8.4 Environment comparability: 防止环境变化伪装成算法进步

比“实验跑不了”更危险的是环境变化让结果**看起来变好/变差**。例如 TensorRT cache、simulator tick rate、physics version、CUDA/kernel、模型权重缓存、外部 API version 都可能成为 confound。

Gate-2 应额外检查 environment comparability:

- comparison 的关键 runtime/environment 是否一致；
- 若环境发生实质变化，是否重新 baseline；
- environment delta 是否被记录；
- 不能比较的 evidence 不做直接数值归因。

无需每轮把 pip freeze 塞进 context。由脚本记录 compact fingerprint/hash；只有重要 delta 写入

Evidence:

```
{
  "schema_version": "1.0",
  "kind": "environment_change",
  "change": "simulator physics 30Hz -> 60Hz",
  "comparability": "requires_rebaseline",
  "affected_capabilities": ["control_latency", "collision_dynamics"]
}
```

9. 跨 Codex / Claude Code 的 Skills 架构

截至 2026-09，Codex 原生支持 repository-local skills 位于 .agents/skills，Skill 由 SKILL.md + optional scripts/references/assets 组成，并使用 AGENTS.md 作为项目级 instructions。[S1][S2] Claude Code 原生支持 .claude/skills/<name>/SKILL.md 和 CLAUDE.md；Claude Code 还可以在 CLAUDE.md 中使用 @AGENTS.md 导入同一份项目指令，从而避免复制。[S3][S19]

因此本方案的跨客户端目标不是模拟两套 CLI，而是构建一个**最低公共能力面（Native Compatibility Contract）**。

9.1 Native Compatibility Contract

V0/V1 Common Core **只允许依赖**：

- repository instructions
- + Agent Skills (SKILL.md)
- + ordinary file read/write
- + local shell / Python
- + Git

这五项足以完整实现 Research Engineering。以下能力不得成为正确运行的前提：

- hooks
- subagents
- MCP
- custom daemon/server
- workflow engine
- cloud agent runtime

GitHub API

client-specific frontmatter

它们可以是 optional accelerators, 但删除它们后 research semantics 必须保持不变。

客户端差异只保留在两个地方:

能力	Codex	Claude Code
Repo Skill	.agents/skills/<name>/SKILL.md	.claude/skills/<name>/SKILL.md
主调用	\$research-engineering	/research-engineering
Project instruction	AGENTS.md	CLAUDE.md; 推荐正文仅 @AGENTS.md + Claude-specific optional notes

因此推荐把 AGENTS.md 作为 common project instruction source, CLAUDE.md 原生导入它; Skills 则维护一份 canonical skills/, 通过本地 installer copy 到两个客户端目录。Canonical Skill 不使用 client-specific frontmatter。

9.1.1 Instruction Surface Budget: 项目指令只做路由, 不承载整套协议

Codex 会把 AGENTS.md 直接注入任务上下文, 并对合并后的 project instructions 有大小上限 (当前官方默认 project_doc_max_bytes=32 KiB) ; Claude Code 也会在 session 启动时加载 CLAUDE.md 及其 import。[S2][S19] 因此不能把本设计全文复制进 AGENTS.md/CLAUDE.md。

项目 instruction 文件只保留: 角色、最高优先级 invariants、Skill 路由、HARD safety/permission 规则、语言规则和“Resume first”等少量常驻约束。Environment/continuity/evidence/evaluation 等详细协议放进对应 SKILL.md / references/, 按需加载。这样既减少常驻 context 成本, 也避免同一规则在多个文件重复后 drift。

9.2 推荐目录

project/	
—— AGENTS.md	# common AI project contract
—— CLAUDE.md	# normally: @AGENTS.md + optional Claude notes
—— skills/	# canonical portable skill source
—— research-bootstrap/	# top-level zero-state initialization

```

| | └── SKILL.md
| ├── research-engineering/
| | ├── SKILL.md
| | └── references/
| | ├── architect-signals.md
| | ├── evidence-model.md
| | ├── environment-feasibility.md
| | ├── session-continuity.md
| | ├── git-research-infrastructure.md
| | └── minimum-correctness.md
| ├── research-status/           # top-level read-mostly Project Working Model
| ├── research-diagnosis/
| ├── hypothesis-design/
| ├── research-search/
| ├── evaluation-design/         # evaluation surface / metric research
| ├── experiment-review/
| ├── research-retrospective/
| ├── scenario-redteam/
| └── integration-promotion/
└── .agents/skills/              # generated copy for Codex
└── .claude/skills/              # generated copy for Claude Code
└── research/
| ├── ACTIVE.json                # execution pointer / write-ahead state
| ├── CURRENT.md                 # compact research working memory
| ├── ARCHITECT.md
| ├── BOUNDARIES.md
| ├── ENVIRONMENT.md
| ├── FINDINGS.md
| ├── EVIDENCE.jsonl             # V0 single-writer ledger
| ├── STATUS.md                  # optional derived human snapshot, not canonical
| ├── runs/
| | └── EXP-*/manifest.json
| └── ledger/                     # V1 immutable sharded records
└── research_probes/

```

```

|—— tools/
|   |—— install_research_skills.py
|   |—— researchlog.py          # optional deterministic helper
|—— src/

```

9.2.1 Common Core 与 Optional Adapter

Common Core (必须跨客户端一致) :

- SKILL.md 语义;
- AGENTS.md common contract;
- research state + ACTIVE;
- Git/branch/worktree/checkpoint 规则;
- local shell / Python probes;
- Evidence schema;
- Architect Signals;
- Research / Integration mode;
- Minimum Correctness Envelope;
- Environment-Constrained Research Protocol;
- Session Continuity & Resume Protocol;
- research-bootstrap zero-state initialization contract;
- research-status Project Working Model synthesis contract;
- evaluation-design evaluation-surface research contract;
- versioned/crash-safe state contract。

Optional Adapter (不可成为语义依赖) :

- Claude Code hooks / subagents;
- Codex hooks/subagents/parallel features;
- client-specific permission profiles;

- GitHub remote / Actions / Environments / PR / Releases;
- IDE/UI integration。

真正的 evaluator protection 不应只依赖 hooks；可信边界优先由文件权限、独立 checkout、container、独立 OS user 或 official judge 建立。

9.3 Skill-enforced Research Artifact Language Policy

这里的语言规范不是对“整个代码仓库”的统一语言要求，而是 **research-engineering 在研究过程中生成 artifacts 时必须遵守的输出路由规则**。其目的有两个：让架构师获得高密度、自然的中文研究材料；让 AI 的 durable state 与 machine-readable records 在 Codex / Claude Code 间保持稳定、低歧义。

- **Human-facing research artifacts：中文。** 包括 Research Brief、checkpoint summary、architecture review、diagnosis summary、experiment review、retrospective、方案比较、阶段总结、比赛分析、research-status 生成的 Project Working Model，以及明确面向架构师/团队阅读的说明。技术术语、算法名、库名、协议名、metric 名和专有名词保留英文原词，例如 Evidence Ladder、vertical slice、Gate-2、Kalman filter。
- **AI-facing research artifacts：英文。** 包括 SKILL.md / skill references、AGENTS.md / CLAUDE.md 中的 AI instructions、CURRENT.md、ARCHITECT.md、BOUNDARIES.md、ENVIRONMENT.md、FINDINGS.md、hypothesis/evidence/failure records、schema、JSON/JSONL、agent handoff、machine-facing hooks/scripts message。
- **CLI 对架构师的可见 research update 默认中文。** 不要因为内部 state 使用英文，就把日常研究汇报变成英文。
- **Architect input 可保持中文。** 当 OBSERVE / SUSPECT / DIRECTION 等信号持久化到 AI-facing state 时，保存英文规范化语义；仅当原始措辞本身对 diagnosis 或物理观察有意义时，再附带原始 source_text。
- **Mixed-audience artifact 不做中英双份正文。** 优先拆成“中文 human summary + 英文 AI state/record”，中文 summary 用 evidence/hypothesis ID 指向机器记录，避免双份事实源产生 drift。
- **Source code 不由本规则强制翻译。** 代码 identifier、comment、docstring 遵循项目已有约定；若项目没有约定，默认使用英文。外部原始资料保持原语言，需要给架构师总结时另生成中文摘要。

因此，这是一条 **Skill runtime contract**：同一次 research iteration 可以同时产生英文 EVIDENCE.jsonl record 和中文 Research Brief；二者服务不同 audience，而不是相互复制。

Optional Adapter（不可作为协议依赖）：

- Claude Code hooks：可用于自动记录或阻止特定工具行为；
- Claude subagents：可把 retrospective/literature search 放到隔离 context；
- Codex subagents/并行功能：可在 V1/V2 用于 read-heavy search；
- 客户端-specific permission profiles；
- UI / IDE integration。

真正的 evaluator protection 不应依赖 hooks；可信边界应该靠文件权限、独立 checkout、容器、独立 OS 用户或官方 judge。

10. research-engineering 主 Skill 设计

主 Skill 是**默认行为模式**，不是每轮工作流。它的职责是解除 Coding Agent 的生产工程默认值，并建立 Research Engineering 的权责和证据纪律。

10.1 主 Skill 必须做到

- 从只有大需求的 Day 0 也能工作；若 canonical research state 尚不存在，先进入 research-bootstrap，不得伪造 Resume 状态；
- **每个新 session 先执行 Resume Protocol，再选择新任务；**
- 自动读取 ACTIVE / CURRENT / ARCHITECT / BOUNDARIES / ENVIRONMENT / FINDINGS；
- 用 Git HEAD/status/diff/run manifest 与 ACTIVE 对账，不能相信聊天摘要；
- 把架构师普通技术评论按 signal 语义处理；
- 自己决定 routine 算法与实现；
- 不把 uncertainty 变成用户选择题；
- 主动寻找 executable evidence；
- 在实验不可行时区分 environment limitation 与 scientific negative；

- 执行 Surrogate Validity Contract, 不让“能跑的 mock”冒充有效 evidence;
- 自主 reshape/decompose experiment, 必要时构造 research harness;
- 识别 environment confound, 并在 material change 后使用 anchor rebaseline;
- 强制区分 observation / interpretation;
- 不默认调用所有子 skill;
- 不默认 TDD、plan、full regression;
- 有证据时允许推翻 provisional architecture;
- 在架构师纠偏后自动恢复 autonomy;
- 用 Git commit/diff/branch 保证 research provenance;
- substantial batch 或主动 Session Rotation 前留下 compact durable state;
- interrupted session/run 不得被误判为 scientific failure;
- 每个 autonomous research block 遵守显式 Block Contract, 防止无界 exploitation;
- 评价面缺失/失真时把 evaluation 本身视为 Research Subject。

10.2 建议的 Research Block

用户常用:

\$research-engineering

继续 CURRENT.md 中的研究。

最多进行 5 个 evidence-producing iterations,
或在一个重大技术不确定性得到清晰结论时结束。

routine 技术决策自行完成。

只有 HARD boundary、重大架构裁决、外部费用/权限、
官方提交或物理安全边界需要找我。

这里“5 个 iteration”不是 5 次 edit, 而是 5 条**改变技术判断的有效证据**。

10.3 Research Block Contract: 长时自治必须有边界

Research Mode 高自治不等于无限运行。每次较长 autonomous batch 应在 ACTIVE.json 中维护一个轻量

Block Contract:

```
{
  "block": {
    "id": "RB-024",
    "objective": "Resolve recovery oscillation",
    "max_evidence_iterations": 6,
    "max_wall_clock_minutes": 180,
    "completed_evidence_iterations": 2,
    "stop_conditions": [
      "question_resolved",
      "hard_boundary",
      "major_architecture_decision",
      "no_valid_evidence_path"
    ]
  }
}
```

Block Contract 是 autonomy guard, 不是项目计划:

- max_evidence_iterations 统计 belief-changing valid evidence, 不把 INFRA_FAILED 等故障硬算成研究进展;
- 到达边界后应 synthesize 当前 belief, 再决定新 block, 而不是机械继续 30 个相似 mutation;
- 架构师可显式覆盖 block limits;
- routine technical choices 仍由 AI 自主, 不因存在 block contract 而逐步请示。

11. 核心入口 Skills 与专业专家 Skills

三个一级入口 Skill 分别覆盖项目生命周期的三个基本入口: research-bootstrap 回答“新项目怎么进入可研究状态”, research-engineering 回答“现在继续研究什么并实际执行”, research-status 回答“整个项目当前是什么状态”。其余 Skills 是按认知任务触发的专业工具, 不形成固定 pipeline。

11.1 research-bootstrap: 从普通 Repo 到可运行 Research State

research-bootstrap 只在 canonical research state 缺失、损坏到无法恢复，或架构师明确要求重新初始化研究基线时使用。它不是 brainstorming/project-planning skill。

它应完成：

inspect problem / rules / repo / SDK / data / baseline

- identify external truths and HARD boundaries
- discover current Research Environment
- separate known facts / assumptions / unknowns
- identify highest-value technical uncertainties
- create minimal canonical research state
- record initial E0 Evidence where justified
- choose first executable evidence target
- hand off to research-engineering

默认创建/初始化：ACTIVE.json、CURRENT.md、ARCHITECT.md、BOUNDARIES.md、ENVIRONMENT.md、FINDINGS.md、Evidence ledger 和必要的 runs/ 目录。

它禁止：一开始就设计完整最终架构、生成几十个 task、建立重型测试矩阵、把 provisional idea 写成 HARD contract、或在没有必要 Evidence 的情况下大规模实现。Bootstrap 成功的标准是**第一个技术不确定性已经可以被执行性证据推进**。

11.2 research-status: 总架构师的全局 Project Working Model

这个 Skill 不负责计划下一轮研究，也不是传统项目管理式统计“完成了多少 task”。它把分散在 repo 中的 durable state 编译成一份**高密度、证据可追溯、同时能被 AI 重新吸收的项目状态快照**。

读取面：

ACTIVE

+ CURRENT

+ ARCHITECT

+ BOUNDARIES

+ ENVIRONMENT

+ FINDINGS

+ referenced Evidence / run artifacts
 + Git HEAD / branch / status / relevant research commits/tags
 (+ optional GitHub remote/verifier state)

默认报告回答：当前在造什么、已经证明什么、哪些仍 provisional/refuted/open、三维 maturity、当前 bottleneck、active experiment、Research Environment 限制、research frontier、下一阶段重点、真正需要总架构师决策的事项。

双重用途但单一事实源

research-status 的可见输出主要给总架构师，因此按语言协议使用中文，但保留 EV-* / H-* / EXP-* / FND-* / Git SHA / tag 等 canonical identifiers，并采用稳定结构。这样同一份报告也可以被粘贴给新的 Codex/Claude Code 作为 **fast cognitive rehydration context**。

但它绝不能替代正式 Resume Protocol：

research-status = fast cognitive rehydration
 Session Resume = authoritative operational recovery

开始修改代码前，新 Agent 仍必须核验 ACTIVE ↔ Git ↔ run artifacts ↔ Evidence。

Read-mostly contract

默认不得：修改 application code、启动实验、改变 hypothesis/FINDINGS、创建新 research branch、promotion、reset/clean Git。若发现状态冲突，只报告 STATUS INTEGRITY WARNING，由 research-engineering 做 reconciliation。

默认汇报结构

1. 项目一句话状态
2. 当前工作契约
3. 三维成熟度：System / Evidence / Research Environment
4. 当前系统与架构形态
5. 近期实质进展（只报 belief/architecture-changing progress）
6. 当前 Findings：Established / Provisional / Refuted / Open
7. Current Research Frontier（有多个有效方向时）
8. 当前关键 Unknowns / Failures / Bottlenecks
9. Research Environment（只报影响当前研究的能力/限制）
10. 当前 Active Research

11. 下一阶段研究重点
12. 需要总架构师关注或决策（默认：无）
13. 风险与偏航提示
14. 继续研发上下文摘要
15. Snapshot basis / provenance

默认控制在约 1–3 屏。详细 evidence history 只有架构师明确要求才展开。

STATUS.md 只允许作为 cache

默认 \$research-status / /research-status 只在 CLI 显示。只有架构师明确要求“保存当前状态”或 milestone/checkpoint 需要时才写 research/STATUS.md，且文件头必须标记：

DERIVED SNAPSHOT — NOT SOURCE OF TRUTH

日常查询不应为了生成 status 制造 Git dirty change。

11.3 research-diagnosis

适用：主要是 E3/E4 以后复杂行为 failure。

触发：

- 多层都可能解释 failure；
- 已尝试多个 fix 仍失败；
- timing/recovery/状态传播复杂；
- 相似 case 一些成功一些失败；
- 安全 top event 有多个 AND/OR 条件。

吸收：Superpowers systematic-debugging 的 evidence-before-fix、boundary instrumentation、backward tracing；LifeOS 的 Fault Tree / Kepner-Tregoe method routing；agent incident postmortem 的 trace reconstruction 与 earliest preventable divergence。[S4][S6][S7]

核心输出不是 fix，而是：

first divergence

+ earliest preventable divergence

+ competing hypotheses

- + supporting/contradicting evidence
- + cheapest discriminating test

允许 INCONCLUSIVE + high confidence。

11.4 hypothesis–design

适用于真正存在多个机制/算法/架构解释时。每个 hypothesis 至少包含：

- falsifiable claim；
- mechanism；
- predictions；
- guardrails；
- falsifier；
- cheapest discriminating evidence；
- rough cost。

AI 要自己 rank 并选择 routine next experiment，而不是把 A/B/C 抛给架构师。

11.5 research–search

这个 skill 用于“算法路线不明确，需要外部技术知识”而不是一般 Web 搜索。它应：

1. 先明确当前技术 unknown；
2. 搜论文/源码/benchmark/issue，而不是泛泛找“十大方案”；
3. 优先可运行开源 reference；
4. 提取可测试 mechanism，而不是复制完整框架；
5. 输出 2–4 个值得做 executable confrontation 的方向；
6. 尽快回到运行，不无限 research 文献。

11.6 evaluation-design: 把“怎么判断好坏”本身当成研究对象

当没有可信 evaluator、local metric 与真实行为冲突、official judge 昂贵/稀疏、或人类 qualitative observation 尚未被 operationalize 时使用。它的目标不是随便加 metric，而是建立**能回答当前 research objective 的 Evaluation Surface**。

典型职责：

objective / rubric / human observation

→ observable behavior

→ failure / success dimensions

→ metric candidates + qualitative signals

→ scenario / slice / replay design

→ dev vs holdout separation

→ proxy limitations

→ calibration against E4/E5 / human observation

必须明确：

- metric 能测什么、不能测什么；
- 是否存在 gaming / leakage / proxy overfit；
- clean negative / hard negative / representative scenarios；
- evaluation data 与 training/tuning data 的隔离程度；
- local evaluator 对 E4/E5 的已知 correlation / uncertainty；
- 若评价面变化是 material change，旧结果是否需要 rebaseline。

它可以产生新的 evaluation_surface Research Subject、probe、scenario suite 或 verifier contract，但不能把一个未经校准的新 scalar 直接升级成“真实目标”。

11.7 experiment-review

适用于 meaningful run 后，分四层：

OBSERVATION

→ COMPARISON

→ INTERPRETATION

→ DECISION

并独立记录：

execution_status: ran / invalid / infra_failed / budget_truncated

research_outcome: confirmed / refuted / inconclusive /
failed / informative_failure / promising

confidence: low / moderate / high / certain

这借鉴 Open-AutoResearch 和 lab-notes: infra failure 不能污染 scientific belief, outcome 与 confidence 也不是一个轴。[S8][S9]

11.8 research-retrospective

只在 plateau/阶段节点运行。职责是发现：

- 搜索空间是否过窄；
- proxy 是否替代了真实目标；
- provisional assumption 是否变成了 dogma；
- dominant failure 是否已经转移；
- 是否需要 architecture spike；
- 下一阶段 research budget 应如何分配。

MLEvolve 的 branch diversity、stagnation detection、cross-branch fusion 是重要启发，但不把其 ML runtime 直接搬进 V0。[S12]

11.9 scenario-redteam

当前机制看起来 promising 时主动找 counterexamples：

- timing 极端；
- slow drift / abrupt fault；
- intermittent/freeze/delay；
- 多通道组合；

- recovery-phase attack;
- hard benign negative;
- distribution shift;
- attack without obvious disagreement。

必须同时保留 clean controls; generated scenarios 不应无脑变成训练集。

11.10 integration-promotion

只有被证明值得长期保留时触发：

- 清理 prototype;
- 稳定接口;
- 补关键测试;
- full/representative regression;
- clean-checkout reproduction;
- 删除临时 instrumentation 或正式化;
- 必要的 ablation;
- 把新 baseline 重新交给 Research Mode。

12. 轻量持久状态与 Session Continuity

设计目标不是保存聊天，而是保证 **Semantic / Operational Lossless Recovery**：只要 Git repo、research artifacts 和外部 artifacts 仍存在，新 Codex / Claude Code session 即使看不到旧 conversation，也能恢复全部 decision-relevant research state，并选择与上一 session 连续的下一动作。

需要持久化的不是 chain-of-thought，而是：

research intent

+ architect steering

+ boundaries

+ environment capability

+ current technical uncertainty

- + live hypotheses
- + active experiment intent
- + exact code/worktree state
- + execution progress
- + observed evidence
- + unresolved interpretation
- + next empirical action

V0 核心状态建议七个文件：

research/

```

|—— ACTIVE.json
|—— CURRENT.md
|—— ARCHITECT.md
|—— BOUNDARIES.md
|—— ENVIRONMENT.md
|—— FINDINGS.md
|—— EVIDENCE.jsonl

```

这些是 **AI-facing artifacts**，**正文和结构统一使用英文**；给架构师看的 Research Brief / checkpoint summary 由 AI 另行以中文生成。

12.1 ACTIVE.json: Session Continuation Capsule

CURRENT.md 解决“研究到哪里”，ACTIVE.json 解决“此刻正在做什么”。它必须短、机械、可频繁重写，不保存长推理。

典型字段：

```

{
  "schema_version": "1.0",
  "session_epoch": "SE-20260910-014",
  "status": "running",
  "research_question": "Does motion-aware release reduce recovery oscillation without delaying
safe stop?",
  "subject": {"type": "component", "id": "recovery-policy"},
  "hypothesis_ids": ["H-037", "H-039"],

```

```

"chosen_hypothesis": "H-037",
"experiment_id": "EXP-0142",
"intent": "Add motion-aware recovery release while keeping detection and navigation unchanged.",
"expected_evidence": {
  "supports_if": "oscillation decreases while attack-stop latency is unchanged",
  "weakens_if": "oscillation persists despite correct release timing"
},
"block": {
  "id": "RB-024",
  "objective": "Resolve recovery oscillation",
  "max_evidence_iterations": 6,
  "max_wall_clock_minutes": 180,
  "completed_evidence_iterations": 2,
  "stop_conditions": ["question_resolved", "hard_boundary", "major_architecture_decision", "no_valid_evidence_path"]
},
"git": {
  "branch": "research/recovery",
  "base_commit": "83ab21...",
  "checkpoint_commit": null,
  "dirty_expected": true,
  "expected_touched_files": ["src/safety/recovery.py", "research_probes/recovery/replay.py"]
},
"execution": {
  "status": "partial",
  "completed_cases": ["case17"],
  "pending_cases": ["case21", "case31", "case37"],
  "run_manifest": "research/runs/EXP-0142/manifest.json"
},
"current_observation": "case17 oscillation reduced; evidence is insufficient for a conclusion",
"next_action": "Run remaining replay cases before modifying the mechanism again.",
"updated_at": "2026-09-10T19:28:41+08:00"
}

```

status 建议: idle / planning_evidence / implementing / ready_to_run / running / reviewing / blocked / interrupted。

12.2 Write-Ahead Research State

不能只依赖“session 结束前写总结”，因为 context exhaustion、Ctrl-C、terminal crash 和进程重启可能没有优雅退出。

因此重要动作采用 write-ahead 顺序：

```
choose experiment
→ write ACTIVE intent
→ modify code
→ ACTIVE=ready_to_run
→ create run manifest
→ ACTIVE=running
→ execute
→ append Evidence
→ ACTIVE=reviewing
→ update CURRENT/FINDINGS if justified
→ ACTIVE=idle or next experiment
```

研究动作发生之前先留下 intent，保证任意中断点都可重构。

12.3 Run Manifest: 未完成实验也必须有身份

每个 meaningful experiment 在执行前建立：

```
research/runs/EXP-0142/
├── manifest.json
├── stdout.log
├── stderr.log
├── artifacts/
└── result.json      # only after completion
```

manifest.json 至少记录 schema_version、experiment ID、question、hypothesis IDs、environment ID/fingerprint、Git HEAD/diff、command/config、input identities、start time、status。

manifest.status=running 且 result.json 不存在表示 interrupted/incomplete，不是 scientific negative。

12.3.1 Long-running Experiment Ownership

训练、仿真、benchmark 可能持续几十分钟到数小时，session rotation 不应等价于杀死实验。对 long-running run, manifest 的 execution 至少应可记录：

```
{
  "execution": {
    "host": "workstation-a",
    "launcher": "local_process",
    "pid_or_job_id": "48123",
    "started_at": "...",
    "stdout": "research/runs/EXP-0142/stdout.log",
    "stderr": "research/runs/EXP-0142/stderr.log",
    "heartbeat_or_last_observed_at": "...",
    "expected_outputs": ["artifacts/metrics.json"]
  }
}
```

Resume 时先判断 job/process 是否仍存活，再决定 attach/observe / finalize / mark interrupted / rerun。PID 只在同 host 有意义；Slurm、tmux、Docker、remote runner 等可以通过 launcher + job_id 扩展，但都不是 Common Core dependency。不得因为旧 session 消失就重复启动同一个昂贵 run。

12.4 Session Resume Protocol (Normative)

每一个新的 \$research-engineering / /research-engineering session **在提出新 idea 前必须先恢复现场**：

1. Read ACTIVE.json, CURRENT.md, ARCHITECT.md, BOUNDARIES.md, ENVIRONMENT.md, relevant FINDINGS.md。
2. Inspect git rev-parse HEAD, current branch/worktree, git status --short, relevant git diff。
3. 如果 ACTIVE 非 idle，先重构 active experiment，不得另起 research task。
4. 读取 active run manifest 和已经产生的 artifact/result。
5. 只按 CURRENT/FINDINGS/ACTIVE 引用读取 recent/relevant evidence；**不要每次重新加载整个 ledger。**
6. 对 ACTIVE ↔ Git ↔ run artifacts ↔ Evidence 做 reconciliation。

7. 不得静默丢弃 unfinished work、unexpected dirty diff、pending run 或 architect constraint。
8. 恢复后决定 resume / rerun / review / abandon / start-next。
9. 只有发现 material ambiguity、冲突或需要架构师决策时，才给人一个中文 resume note。

Reconciliation 规则

ACTIVE expected dirty == Git dirty

→ continue

ACTIVE says idle but Git dirty

→ RECOVERY_RECONCILIATION before new research

ACTIVE points to experiment but branch/worktree differs

→ locate correct worktree or mark ambiguity; do not execute blindly

manifest running + result absent

→ interrupted; inspect process/artifacts, then resume or rerun

12.5 Session Rotation

长时间研究应把 session 视为 disposable compute context，而不是努力把一个 context 撑到极限。

建议在以下语义边界主动 rotation：

- 一轮 evidence iteration 已干净结束；
- major branch/direction 刚改变；
- retrospective 刚完成，working model 已重写；
- raw logs/code discussion 已显著污染 context；
- Agent 出现重复重建上下文、混淆旧假说或引用失准。

Rotation 前不写长 handoff 文档，只需更新 ACTIVE + CURRENT + Evidence；如有重要 dirty state，建立 checkpoint commit。

12.6 Interrupted execution semantics

Session/client interruption 与 infrastructure/scientific failure 分离。Execution status 建议：

completed
 interrupted
 infra_failed
 env_blocked
 env_unsupported
 resource_exceeded
 invalid

interrupted 的 reason 可为 session_boundary / client_restart / process_signal / machine_shutdown / unknown。它不更新 scientific belief。

12.7 CURRENT.md: Research-level working memory

只保留：Objective、Evidence maturity、Research environment maturity、Working pieces、Provisional system shape、Highest-value uncertainties、Active research、Current frontier、Next empirical action、当前 block 的 limits/escalation。应控制在新 session 能快速读完的规模；不要变成 timeline。

12.8 ARCHITECT.md: 当前有效的人类架构信号

保存 Active Direction / Observations / Suspicions / Decisions / Constraints(scope+expiry) / Vetoes / Open strategic questions。它保存当前仍有效 steering，不保存完整聊天历史。Resume 时必须重新计算 expiry，既不能忘掉临时约束，也不能让其永久化。

12.9 BOUNDARIES.md

HARD：赛题规则、外部数据政策、official metric semantics、已验证 SDK 坐标/单位、physical safety envelope、judge credentials、预算上限。改变必须找架构师。

PROVISIONAL：WorldState、模块接口、safety/planner boundary、当前 replay 格式等。AI 可以基于 evidence 改，但必须记录。

FREE：算法、模型、阈值、内部结构、prototype、probe、instrumentation、synthetic generators、低成本 local harness。

12.10 ENVIRONMENT.md: 当前实验能力边界

记录“实验室有哪些仪器”，不是 architecture requirements: Available compute/devices、Simulator/SDK、Replay/Data/External Services、Known Limitations、Available Substitutes、Research Capability Map、Important fingerprints/deltas。

每项重要 capability/limitation 建议带 verified_by: EV-*. 重要 material change 标记 COMPATIBLE / REBASELINED / INCOMPARABLE。

12.11 FINDINGS.md

只写 durable beliefs: Established / Provisional / Refuted / Open contradictions。每条引用 Evidence IDs；不能写“之前好像试过”。FINDINGS 是 context compression，不是日志。

12.12 EVIDENCE.jsonl

V0 单 writer 时是 raw source-of-truth；每条 Evidence 保存 question、subject、evidence level、observations、measurements（可空）、supports/contradicts、limitations、environment provenance、Git provenance、artifacts、confidence。

12.13 STATUS.md: 可选 Derived Snapshot

STATUS.md 不是新的 durable source-of-truth，而是 research-status 在架构师明确要求保存、阶段 checkpoint 或 milestone 时生成的中文快照。它必须引用 canonical IDs 和 snapshot provenance，并在文件头标注 DERIVED SNAPSHOT — NOT SOURCE OF TRUTH。

默认日常状态查询只显示 CLI 输出，不修改 STATUS.md，避免 status 查询本身制造 Git dirty state。过期 STATUS.md 不能覆盖 CURRENT/FINDINGS/ACTIVE/Git；新 Agent 可用它快速建立全局认知，但开始执行前仍必须走 Resume Protocol。

人类物理/仿真观察也作为 evidence 进入 ledger；AI 应尽量把高价值 qualitative observation operationalize 成可重复 probe/metric，但不能因为暂时没有数字就丢弃。

12.14 V1: 并行后改成 sharded immutable ledger

多个 worktree/agent 并行时，不要同时 append 一个 JSONL：

research/ledger/EV-001.json

research/ledger/EV-002.json

...

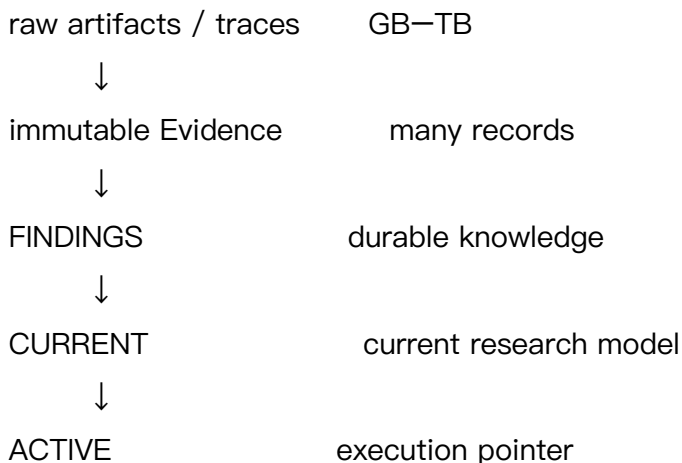
EVIDENCE.jsonl # generated view only

CURRENT.md、ARCHITECT.md、BOUNDARIES.md、ENVIRONMENT.md 保持明确 single-writer ownership；worker worktree 主要写代码、run artifacts 和 immutable evidence records。

并行后 ID 不能依赖无锁全局自增计数器。EV-* / EXP-* / H-* 至少对并行 writer 使用 collision-resistant ID，例如 EV-20260911T101530Z-a7f3、ULID/UUID，或等价的时间+随机后缀方案。禁止多个 worktree 通过“读取最后一个 EV 编号再 +1”分配 ID；否则 merge 时会产生语义碰撞。

12.15 Context Compression Hierarchy

跨 session 恢复应遵循分层读取，而不是重放全部 history：



新 session 从最上层小状态向下按引用展开，只有冲突或 diagnosis 时才下钻 raw evidence/artifacts。

12.16 Session Recovery Benchmark

V0 就应做“失忆测试”：Session A 在 active experiment 中途结束；Session B 不提供旧聊天，只调用 research-engineering。检查：

- 是否找到正确 branch/worktree/HEAD；
- 是否理解 dirty diff 的意图；
- 是否知道当前 hypothesis / experiment；
- 是否知道 completed/pending cases；

- 是否保留 architect signal 和 environment limitation;
- 是否避免重复已经完成的实验;
- 是否选择正确 next action。

新增 KPI: Session Recovery Accuracy、Time-to-resume、Duplicate-work-after-rotation rate。

12.17 State Schema Versioning 与 Crash-safe Canonical Writes

所有 machine-readable canonical artifacts (至少 ACTIVE、run manifest、Evidence record、environment-change record; 后续独立 hypothesis record 同理) 必须带 schema_version。

兼容规则:

known current schema → read/write normally

known older schema → migrate conservatively, preserving unknown data

unknown newer schema → DO NOT overwrite; report incompatibility

invalid/partial JSON → recover from Git/checkpoint/temp file if possible; otherwise reconcile

任何 Skill 都不得“猜测”未知字段含义后静默重写。schema migration 是 deterministic state maintenance, 不是研究判断。

对 ACTIVE.json 等恢复关键文件, 写入应采用 crash-safe replace:

write <file>.tmp

→ close / flush

→ validate syntax + required fields

→ atomic rename/replace canonical file

researchlog.py 若存在, 应封装这套写入; 若 V0 不实现 helper, Codex/Claude Code 仍需遵守“先完整写临时文件并验证, 再替换”的语义。目标不是建立数据库事务, 而是避免 context crash / process kill 留下半个 JSON 使下一 session 无法恢复。

13. 可选本地工具: researchlog.py, 不是 orchestrator

上一版 researchctl.py 的命名容易误导为“中央控制器”。建议 V0 改为:

tools/researchlog.py

它只做 deterministic bookkeeping:

init 建立空的 versioned research state skeleton (不做研究决策)
 record 写一条已有 evidence (包括 E0 和 architect observation)
 run 执行任意 shell command, 抓 stdout/stderr/duration/git/input provenance
 compare 比较两条 evidence 的公共 measurement / artifacts
 snapshot 输出当前 git/config/environment/input fingerprint
 env 记录重要 environment change / capability delta
 active crash-safe 更新 ACTIVE.json
 manifest 创建/更新 versioned run manifest
 validate 验证 canonical state schema/syntax
 job 检查 long-running local job/process 状态
 reconcile 对 ACTIVE/Git/run/Evidence 做 deterministic consistency report
 checkpoint 建立可恢复 Git checkpoint

它**不做**:

- LLM 调用;
- hypothesis 选择;
- scheduler;
- evaluator abstraction;
- keep/revert policy;
- 自动 promotion;
- 数据库。

如果手写 JSONL 已经足够, V0 甚至可以不实现该脚本。

14. Git / GitHub Research Infrastructure、worktree、evaluator isolation 与可信研究

Git 不只是 source control, 而是 Research Engineering 的 **normative executable-state layer**。Evidence 说明“学到了什么”, Git 说明“产生这些知识时系统究竟是什么”, ACTIVE 说明“此刻做到哪里”。

Evidence = research knowledge / observation

Git = exact executable state / provenance / branch history

ACTIVE = current execution pointer

GitHub 是 **optional but strongly recommended remote research plane**: 用于远端 durability、跨机器恢复、独立 verifier、promotion review、里程碑 artifact。即使没有 GitHub, Common Core 仍必须完全工作。

14.1 Research Transaction: 把 meaningful iteration 映射到 Git

不要每次 edit 都 commit, 也不要一天结束才 commit。推荐粒度是“获得一条 meaningful evidence”或“形成一个安全 session checkpoint”:

research question

→ intervention

→ run / observe

→ evidence obtained

→ research transaction boundary

→ commit + evidence reference

Commit message 轻量规范:

research(EXP-0142): add motion-aware recovery release

Hypothesis: H-037

Evidence: EV-0821

Outcome: promising

Level: E2

失败但 informative 的实验同样可以 commit; 是否最终进入 main 由 promotion 决定。

14.2 Git provenance contract

每条 E1+ evidence 尽量记录:

HEAD commit

parent/base commit

dirty flag

diff hash

branch/worktree

config/model/data/environment hash when relevant

当 evidence 来自 dirty working tree 时，必须能通过 diff hash / expected touched files 重构；高价值或跨 session 实验优先 checkpoint commit。

14.2.1 Git ↔ Evidence Transaction Ordering

必须避免“Evidence record 想记录包含它自己的 Git commit hash”这种自引用。推荐规则：

1. EXP-ID 在实验前分配；需要 Git message 双向索引时，EV-ID 也可预分配，且必须 collision-resistant；
2. Evidence 的 code_state 指向**实际产生该证据的 executable state**：clean run 用 commit SHA，dirty run 用 base_commit + diff_sha256；
3. Git research commit 可以在 message/footer 中引用 EXP-ID / EV-ID；
4. Evidence ledger 记录 code_state，但不要求记录“包含该 Evidence 文件自身的 commit SHA”；若要知道 record 最终在哪次 commit 入库，用 Git history 查询；
5. commit 已成功但 Evidence 尚未 append 就发生中断时，Resume/Reconciliation 根据 EXP/run result/commit footer 补齐记录，不把它当 scientific failure。

这样 Git 和 Evidence 可以双向定位，又不会制造不可实现的 hash 自引用事务。

14.3 Checkpoint commit 优于匿名 stash

git stash 可以临时人工使用，但不能作为正式 Session Continuity 机制。跨 session 的 meaningful unfinished work 优先：

checkpoint(EXP-0142): recovery replay partially complete

并在 ACTIVE 中记录 checkpoint_commit、completed/pending cases。Research branch history 不要求 production-level 整洁；恢复能力优先。Integration Mode 再 squash/clean history。

14.4 Branch 粒度：按 Research Direction，不按每个微实验

micro mutation

→ current research branch

independent/risky mechanism

→ research/<topic>

architecture alternative or parallel write-heavy work

→ research/<architecture-family> + worktree

不要生成 research/H001...H999 的 branch 爆炸。一个 branch 可以包含同一方向的多个 EXP。

Git worktree 原生允许同一 repository 同时 checkout 多个 working tree，适合真正独立且会冲突的研究方向。

[S20]

14.5 Worktree 并行规则

- write-heavy parallel branch 不共享 working tree；
- read-heavy exploration 可不建 worktree；
- worktree worker 不独立重写全局 CURRENT/ARCHITECT/BOUNDARIES；
- worker 输出 immutable evidence + artifacts，由 lead research session synthesis；
- rejected branch 可删除，但 Evidence 不删除。

14.6 Promotion、Baseline 与 Tag

Promotion 的对象是**被验证的 mechanism**，不是整个实验 branch。推荐：

research branch

→ Gate-3

→ architect approves promotion

→ Integration Mode

→ extract minimal validated mechanism

→ cherry-pick / clean implementation

→ main

→ annotated baseline tag

稳定节点用 annotated tag 标识，例如：

baseline/e3-recovery-v2

baseline/system-v0.4

submission/2026-09-21-004

官方 submission 必须绑定 immutable commit/tag + config/model/data manifest, external score 回写为 E5 Evidence, 禁止出现“提交了一个 zip, 但不知道来自哪一版”的状态。

14.7 Git 与 Session Resume reconciliation

Resume Protocol 不能只相信 ACTIVE; 必须核对:

ACTIVE expected state



Git HEAD / branch / status / diff



run manifest / artifacts



Evidence ledger

unexpected dirty tree、branch mismatch、missing manifest 都触发 RECOVERY_RECONCILIATION, 不得静默 reset/checkout 覆盖现场。

14.8 GitHub Remote: 跨机器与灾备

GitHub 不是 canonical state, 但 research/checkpoint branch push 后可以提供 machine-level continuity:

session durability → local repo

machine durability → Git remote

research semantics → versioned research artifacts

建议在 substantial checkpoint、重要 branch、promoted baseline 后 push。不要为每个微小 mutation 强制 push。

14.9 GitHub Actions: 只用于昂贵/独立验证, 不做每 patch CI

Research Mode 禁止把 GitHub Actions 重新变成每 commit 触发的 full regression。它更适合:

- manual/on-demand Gate-3 suite;
- clean environment / cross-machine benchmark;
- protected Gate-4 verifier;
- expensive GPU runner;
- official submission adapter。

优先 workflow_dispatch / CLI 手工触发，而不是每个 research commit 自动跑长 CI。[S21]

示意：

```

local AI-writable branch
    ↓ push / manual dispatch
GitHub Actions verifier
    ↓
metrics / trajectory / artifacts
    ↓
Evidence record
  
```

14.10 GitHub Environments: Architect Gate / credential isolation

official-judge、昂贵 cloud GPU、physical-robot credential 等可以放在受保护 GitHub Environment 中，由 reviewer/approval rule 控制。Research Agent 只负责推荐调用，不持有最终 credential。这样可以把“不要私自 official submit”从 prompt 规则升级成 out-of-band enforcement。[S22]

14.11 PR / Issue / Release 的克制使用

PR: routine research 不要求；只在 promotion / Integration Mode 使用，作为“research artifact → stable component”的审查边界。

Issue: 不复制每个 hypothesis/experiment；只记录长期 architecture challenge、environment capability gap、competition rule ambiguity、external blocker 等需要人/团队持续关注的项目。

Release: 只用于 promoted baseline / competition submission / reproducible milestone，不为普通实验创建。

GitHub 上的信息是 coordination/derived view；canonical research truth 仍是 repo commits + research artifacts + Evidence IDs，避免双 source-of-truth。

14.12 Large artifacts: Git 保存 provenance，不吞掉实验世界

大型 model、dataset、trajectory、video 不直接塞普通 Git。Git 中保存：

artifact:

uri: <local/NAS/object-store/Actions-artifact/...>

```
sha256: ...
size: ...
producer: EXP-0142
```

需要时再使用 Git LFS/DVC/object storage；这些都不是 V0 dependency。

14.12.1 Mutable Input Identity Contract

Git 只能精确标识代码，Applied-AI comparison 往往还受 dataset、replay suite、foundation model/checkpoint、prompt、scenario generator、config 等 mutable inputs 影响。凡参与 causal comparison 的 mutable input 都必须有 stable identity，而不能只写“用了同一套数据”。

推荐在 run/evidence manifest 中记录：

```
inputs:
  dataset: data-v3@sha256:...
  replay_suite: recovery-12@sha256:...
  model: qwen-vl-72b@checkpoint:...
  prompt: prompts/planner-v7@sha256:...
  config: configs/H037.yaml@sha256:...
```

如果某项无法 hash，至少记录可重新定位的 immutable version/URI/commit/tag。一次 comparison 同时改变多个关键 input 时，不得把 delta 单独归因给算法。

14.13 Research Environment provenance 与 confound control

任何会影响实验结论的环境因素都应可追溯，但不把完整 dependency dump 灌入 Agent context。researchlog snapshot 可自动记录 compact environment fingerprint；material changes 追加 environment_change evidence，并按 COMPATIBLE / REBASELINED / INCOMPARABLE 处理。

14.14 Evaluator isolation 随成熟度增强

E0/E1 还没有 evaluator 时，保护的是规则、SDK semantics、hidden labels 等外部真相。

到 E4/E5：

Trust Tier 建议

低 BOUNDARIES.md + hash，仅防误改

Trust Tier 建议

中 sibling evaluator checkout + 人拥有的 verify 命令

高 本地 container / 不同 OS user, evaluator/holdout 只读

更强 GitHub protected verifier / architect-controlled holdout

最强 official judge credentials 对 agent 不可见

Open-AutoResearch 的关键价值之一就是明确区分 prompt-level “请不要改 evaluator”和真正 out-of-band enforcement。[S8]

14.15 防 reward hacking

必须防止：candidate 读取 judge-only labels；agent 改 metric 让自己“变强”；synthetic red-team case 逐渐变成过拟合训练集；official leaderboard 被当无限 tuning metric；local proxy 与真实物理行为分离。

Meta-Agent Challenge 的 dev vs held-out verifier 思路是重要参考：agent 可以反复优化开发 artifact，但最终 judge 独立注入。[S15]

15. 与 Superpowers / GSD 的共存策略

15.1 不要全局删除 Superpowers

应该重新定位：

Research Mode

↓ promising mechanism

Promotion Gate

↓

Integration Mode

↓

Superpowers / GSD / selective TDD / regression

↓

Stable baseline

↓

Research Mode

15.2 Research Mode 中保留 Superpowers 的哪些部分

强烈保留：

- reproduce before guessing;
- evidence at component boundaries;
- backward data/control tracing;
- compare working vs broken;
- 一次实验尽量回答一个清晰 causal question;
- 三次以上相似 fix 失败后质疑 architecture assumption。

不要默认继承：

- 每次实现前都 brainstorming/长方案;
- routine research mutation 必须 test-first;
- 每个新函数都单元测试;
- 每轮 full regression;
- disposable prototype 也要求 production-level review;
- 每轮都向人确认 plan。

15.3 实战中如何防自动触发重型技能

在 AGENTS.md / CLAUDE.md 和主 skill 中写明确冲突规则：

When Research Engineering Mode is active, it overrides generic software-engineering workflow defaults for experimental changes. Do not invoke brainstorming, detailed planning, TDD, code-review, or full-regression skills unless the current task is explicitly in Integration Mode or a critical boundary fix requires them.

Systematic debugging investigation techniques remain allowed; its mandatory TDD/final-verification phases do not automatically apply to disposable research mutations.

如果某个全局插件仍频繁强制 TDD，最佳实战方式是：**Research worktree 只安装 research skills + 所需 debugging skills；Integration worktree 再启用完整 Superpowers。**

16. Codex 中的日常实战 workflow

16.0 总架构师随时查看全局状态

日常最常用的只读入口之一应是：

```
$research-status      # Codex
/research-status      # Claude Code
```

它不启动研究，不修改代码，而是从 repo 的 canonical state 与 Git 编译一份中文 Project Working Model。典型用途：

- 一天开始前快速掌握全局；
- AI 自主研究数小时后检查是否偏航；
- 架构师准备给 CHALLENGE / DIRECTION / DECISION 前先建立共同认知；
- Codex ↔ Claude Code 切换时作为 fast context capsule；
- milestone / submission 前生成 checkpoint snapshot。

状态汇报结束后，若无需架构师决策，AI 应明确写“需要总架构师决策：无”，而不是把 routine 技术选择抛回来。

16.1 Day 0: 只有问题，没有系统

真正的零状态项目优先运行 Bootstrap：

```
$research-bootstrap
```

这是一个新的 Applied-AI research project。

检查赛题/需求、repo、SDK、数据、baseline 和本机环境，

只建立最小 Research Engineering state，识别第一个值得执行验证的 technical uncertainty。

不要生成完整系统设计或长 implementation plan。

Bootstrap 完成后进入：

\$research-engineering

这是具身安全比赛。

目前只有赛事说明、仿真 SDK、baseline 和数据。

DIRECTION:

最终希望得到通用安全层，不做六类攻击的 hard-code detector 集合。

先研究资产与关键技术 unknowns；不要先写完整 architecture document。

自行选择最值得首先获得实证的问题，并尽快构造 executable probe。

理想行为：

读规则/SDK/数据

→ 识别 unknowns

→ 选一个高信息低成本问题

→ 写 probe

→ 运行

→ 记录 E1

→ 根据结果继续

16.2 日常“继续当前研究”

\$research-engineering

继续 CURRENT.md 中 active research。

自行推进算法、实现和实验。

最多做 6 个 evidence-producing iterations。

不要为了凑轮数做低价值参数扫描。

16.3 架构师只给物理观察

OBSERVE:

LiDAR deletion 时已经能可靠停车，但攻击结束以后有时 10~15 秒不走。

DIRECTION:

当前优先研究 recovery。

CONSTRAINT:

本 block 不改 navigation planner; expiry=recovery_checkpoint。

AI 应自己定位 trust/recovery 状态、构造 replay、试算法并回到自主 loop。

16.4 及时纠偏: CHALLENGE

CHALLENGE:

暂停当前 trust-score 参数优化。

最近 12 个实验都默认现有 trust architecture 是正确的，但这个前提没有被证明。

重新打开 architecture assumption，解释为什么真正 failure 不是 recovery ownership。

正确 AI 行为：暂停 narrow tuning → retrospective/diagnosis → 新 architecture spike 或判别实验 → 自主继续。

16.5 架构师直接做决定

DECISION FINAL:

必须存在 planner 之后的独立 safety interception boundary。

理由：赛题存在 planner 下游的 control communication tampering。

内部 safety mechanism 可以继续研究。

AI 不再重做“要不要有这个 boundary”的 research，但 boundary 内算法仍自主。

16.6 算法选型不明确

DIRECTION:

cross-modal consistency 有意义，但现在机制太 heuristic。

研究更通用的 trust representation；延迟预算 < 10ms。

AI 应自己比较 Bayesian belief、temporal filter、learned residual、world-model-lite 等，并选择最便宜的判别 experiment；不应问你“选 A 还是 B”。

16.7 Architecture Spike

DIRECTION:

需要判断 safety arbitration 放在 planner 后 sidecar，还是 policy loop 内。

不要靠长篇架构讨论决定；做两个最薄 spike，实际比较信息可见性、拦截覆盖、latency、state ownership、recovery semantics。

16.8 Promising 后自动 red-team

当一个机制连续表现良好，AI 应主动测试：slow drift、intermittent、composition、hard benign negative、recovery-phase disturbance，而不是直接宣布 solved。

16.9 典型场景：理想实验在当前环境不可行

例如 AI 想验证“planner 后 control tampering 是否能被独立 safety boundary 阻断”，但当前 simulator 不能在 planner→actuator 之间注入攻击。

正确行为不是把实验记为失败，也不是立即向架构师求助，而是：

Desired E4 evidence unavailable

→ mark ENV_UNSUPPORTED

→ determine causal feature that must be preserved

→ build E2 replay shim: planner command → tamper → safety gate

→ if useful, extend to E3 partial stack with real planner + mock actuator

→ record limitation: no real dynamics conclusion

→ continue research

只有当升级到 E4 成为当前主要 score ceiling，而且需要修改 simulator plugin / 购买硬件 / 获取新权限时，才以 Research Environment investment 决策升级给架构师。

17. 具身安全比赛：从 Day 0 到完整系统的贯穿示例

下面展示完整生命周期，重点是“系统本身是研究产物”。

17.1 Day 0: 只有赛事说明、SDK、baseline

架构师：

DIRECTION:

做一个攻击无关的安全框架，不想看到每一种攻击一个专门 detector。

先从我们有数据的 LiDAR deletion 入手，但不要形成 LiDAR-specific architecture。

AI E0:

- 读赛事 rubric：检测、定位、阻断、降级、恢复、任务完成是不同评分维度；

- 读 SDK：确认 camera/LiDAR/IMU timestamp、controller interception 点、baseline navigation 接口；
- 得出 unknown map：
 - U1：未知攻击的通用 signal 来自 sensor-specific anomaly 还是 temporal/cross-modal consistency?
 - U2：安全仲裁应该在哪层？
 - U3：如何从不可信状态恢复可信？
 - U4：task/language attack 和 physical sensor attack 能否共享 trust abstraction?

CURRENT.md 此时明确：**不存在完整 safety system；architecture 未冻结。**

17.2 E1：第一个 LiDAR information probe

AI 不搭完整系统，只写：

raw LiDAR replay

→ angular/occupancy temporal residual

→ plot + simple statistics

结果：

- attack onset 2 帧内 residual 明显升高；
- 但 sparse clean scene 也频繁触发。

正确结论不是“调 threshold”，而是：

missing-evidence signal 存在，但 unconditioned temporal deficit 不能区分攻击与合法稀疏场景。

这是一条 informative_failure / provisional finding。

17.3 E1：第二个 cross-modal probe

AI 利用已有 camera obstacle probability + LiDAR support：

camera evidence + LiDAR support

→ disagreement

发现 benign occlusion / geometry change 也产生 disagreement。

于是旧 belief “sensor disagreement == attack” 被 refute。

AI 自主产生新 hypothesis:

需要 ***trust-conditioned temporal world consistency***, 而不是单帧 *disagreement*。

17.4 E2: 形成第一个 Component

建立:

sensor streams/replay

→ trust estimator

→ timestamped trust state

比较:

- fixed temporal persistence;
- Bayesian-ish trust update;
- lightweight learned residual。

当前数据量下 learned residual 不稳定; Bayesian representation 更稳, 先 promotion 到 component-level, 但没有做完整工程化。

17.5 Architecture Spike: 安全层放哪里

AI 做两个薄 spike:

A: planner → sidecar safety → control

B: trust-aware gating embedded in policy loop

E1/E3 evidence 表明:

- B 拥有更丰富语义上下文;
- A 能覆盖 planner 之后的 control tampering;
- 二者不是互斥, 可能需要两级 safety boundary。

架构师根据 threat model 给:

DECISION FINAL:

保留 planner 后的独立 interception boundary。

是否还需要 policy 内 safety context 继续由 AI 研究。

这就是资深经验用于架构 invariant，而不是指定内部算法。

17.6 E3: 第一个 Vertical Slice

真实关键因果路径：

real/synthetic LiDAR

→ real trust estimator

→ provisional world state

→ real safety governor

→ mock planner forward command

→ allow / slow / block

→ recovery state

此时 VLA 和完整 navigation 仍可不存在。

E3 发现：攻击能阻断，但动态障碍离开时 false stop 较多。

AI 研究 motion-aware invalidation，避免继续盲调 threshold。

17.7 Environment-constrained branch: Control tampering 暂时只能做到 E2/E3

此时团队可能已经能完整模拟视觉/LiDAR 攻击，但 SDK 不支持 planner 后的 command corruption。AI 应在 ENVIRONMENT.md 记录该限制，并构造 command replay / interception shim，先验证：

- safety layer 是否能观察 pre/post-tamper command；
- 哪些 integrity features 可以区分正常 planner change 与篡改；
- block/modify 是否会引入明显 latency；
- recovery ownership 如何传播。

这里可以得到 E2/E3 证据，但必须明确：**没有真实 actuator dynamics，因此不能宣称物理安全已验证。**若后续 rubric 表明 control-integrity 成为主要失分，再决定是否投资 simulator plugin。

17.8 第一个 E4 完整仿真

接 baseline navigation 与 simulator:

sim sensors

→ trust/world state

→ planner

→ safety governor

→ controller

→ robot

→ feedback/recovery

首次完整 episode: 无碰撞, 但攻击解除后长时间停止。

架构师观看录像:

OBSERVE:

case 31/37/42 的行为像 stop-go 振荡; 指标没有捕获。

SUSPECT:

问题可能不在 perception 恢复, 而在 recovery state ownership / release condition。

AI:

1. 把人类观察写入 Evidence;
2. trace 重建 perception confidence / fusion / safety state / controller;
3. 发现 sensor trust 已恢复, 但 safety governor hysteresis 仍未释放;
4. 构造 replay metric: state transition frequency + dwell time;
5. 比较几种 recovery policy;
6. 跑 E3 replay, 再跑 E4 小场景。

17.9 AI 开始走偏时的 CHALLENGE

如果 AI 连续做: decay / smoothing / threshold / another smoothing:

CHALLENGE:

暂停 trust-score tuning。

从物理行为看, 机器人不是“不知道风险”, 而是“知道风险后不知道何时恢复”。

重新从 action state transition 和 recovery ownership 解释最近 failure。
这是方向纠正，不是具体实现要求。

AI 应重新打开搜索空间，而不是等待你继续指挥。

17.10 Promotion

当 trust-conditioned world state + recovery policy 在 E2/E3/E4 多次得到支持并通过 limited red-team:

Research Mode

- Gate-3
- Architect agrees to stabilize
- Integration Mode
- selective cleanup / boundary tests / representative regression
- Promoted baseline
- 回 Research Mode 继续 language/control attack 方向

18. 泛化到 RAG / Agent / VLM / 多模态应用

同一协议的价值在于不依赖机器人。

18.1 RAG

早期：

- E0: 数据与查询分布分析
- E1: retrieval probe (不建完整聊天应用)
- E2: chunker/retriever/reranker replay benchmark
- E3: retrieval → context builder → answer slice
- E4: 完整 RAG task suite
- E5: official / user / blind evaluation

架构师观察：“答案看起来总是过度依赖第一个文档”，AI 应先建立 context-attention / citation coverage probe，而不是直接改 prompt。

18.2 Agent / Tool Use

- E1: tool-selection microbench
- E2: trajectory replay + mocked tool responses

- E3: real planner + two tools + fake environment
- E4: complete task sandbox
- E5: held-out benchmark

复杂失败适合 earliest preventable divergence：错误可能起于 planner，也可能 guardrail 已发现但 action layer 仍执行。

18.3 VLM

- E1: vision encoder / crop / OCR signal probe
- E2: image slice benchmark
- E3: perception → reasoning → action slice
- E4: multimodal application scenario

不要为了验证 crop 是否提高 small-object evidence 先搭完整应用。

19. 开源实现复用矩阵

原则是：**复用协议和局部机制，不盲目采用整套 runtime。**

项目 / Skill	直接吸收	V0 不继承	角色	License/状态
Superpowers	reproduction、边界	每次 mutation	research-	MIT；活跃
systematic- debugging	evidence、backtrace、 working-vs-broken	强制 TDD / full phases	diagnosis 基 础	[S4][S5]
LifeOS	Fault Tree、Kepner-Tregoe、	LifeOS	diagnosis	MIT；活跃 [S6]
RootCauseAnalysis	multi-cause、bias resistance	UI/process	method router	
agent-incident- postmortem	trace reconstruction、earliest preventable layer、 nondeterministic replay	每个 incident 永久回归的重流 程	AI 系统 failure diagnosis	以仓库许可为准 [S7]
Open-AutoResearch	immutable evidence、locked judge、status vocabulary、 promotion、out-of-band	完整 1700 行 protocol 一次 性照搬	Evidence / trust model	MIT；pre-1.0 protocol [S8]

项目 / Skill	直接吸收	V0 不继承	角色	License/状态
	enforcement			
lab-notes	append-only、 freeform/rigorous、outcome vs confidence	每实验大量文档	记录 ergonomics	以仓库许可为准 [S9]
Karpathy autoresearch	极紧的 autonomous empirical loop	固定 train.py/ 单 scalar 假设	哲学原型	README 标 MIT；复制前核 验 LICENSE 状 态 [S10]
codex-autoresearch	Git 边界、verify、 keep/revert、state outside chat	greedy scalar keep/revert	局部 controller 参 考	MIT [S11]
MLEvolve	branch diversity、stagnation、 cross-branch fusion	ML-centric full runtime	V2 slow-loop / portfolio	Apache-2.0 [S12]
ShinkaEvolve	population/island search、 Codex/Claude skills、 headless	没有 evaluator 时作为顶层 controller	E2+ scoped optimizer	Apache-2.0； ICLR 2026 [S13]
Microsoft R&D- Agent	Research ↔ Development ↔ feedback 分离	数据/模型中心 的大框架	技术合伙人角 色参考	MIT [S14]
Meta-Agent Challenge	dev vs held-out verifier、 budget、完整 artifact 迭代	双容器 benchmark 全 套	evaluator isolation 参考	Apache-2.0 [S15]
AIDE	code tree / draft-debug- improve	单 Python ML candidate 假设	search-tree 参考	MIT [S16]
MLE-bench	agent research benchmark / evaluator	只适用于 ML engineering	后续基准参考	见仓库 [S17]
Autoresearch Bench	固定时间内 agent 自主实验能力	V0 软件依赖	workflow	2026 新

项目 / Skill	直接吸收	V0 不继承	角色	License/状态
			benchmark	benchmark
			思想	[S18]

19.1 ShinkaEvolve 应该什么时候真正接进来

ShinkaEvolve 明确擅长“已有 evaluate.py + editable program + score”的问题，并已经提供 Codex/Claude skills 与 headless CLI-backed mutation。[S13]

因此最佳位置是：

System-level Research Engineering

↓

识别一个稳定 E2 子问题

↓

例如：固定 replay 上优化 anomaly score / planner heuristic / small kernel

↓

ShinkaEvolve population search

↓

把 top mechanism 返回主 Research Loop

它不是 Day 0 的 Research Director。

20. V0 / V1 / V2 分阶段落地路线

20.1 V0 — 一天可开始: Research Partner

目标：证明 AI 能把架构师的高层观察/方向自主转化成 executable evidence。

只做：

skills/research-bootstrap/SKILL.md	# zero-state initialization
skills/research-engineering/SKILL.md	
skills/research-status/SKILL.md	# 核心只读 Project Working Model
skills/research-diagnosis/SKILL.md	# 可先精简
skills/hypothesis-design/SKILL.md	# 可先精简
skills/evaluation-design/SKILL.md	# 可先精简；无可信 evaluator 时启用

```

research/ACTIVE.json
research/CURRENT.md
research/ARCHITECT.md
research/BOUNDARIES.md
research/ENVIRONMENT.md
research/FINDINGS.md
research/EVIDENCE.jsonl
research/STATUS.md          # optional derived snapshot, default absent/stale-safe
research_probes/
可选 tools/researchlog.py

```

第一天验收：

- Codex 与 Claude Code 都能进入 Research Mode；
- 没有 runnable system 也会自主做 probe；
- 不默认写长 plan 和大量 tests；
- AI 能连续做 2—3 个 evidence-producing iterations；
- 架构师不指定具体算法；
- 新 session 只靠文件能恢复研究状态；
- 一个明确不可行的实验会被分类为 environment limitation，而不是 scientific failure；
- AI 能自行构造至少一种合法 surrogate / harness 继续获得 evidence；
- 全新 repo 可通过 research-bootstrap 在不写重型 plan 的前提下建立最小 canonical state；
- 缺失/失真的 evaluator 能通过 evaluation-design 被明确当成 Research Subject；
- canonical JSON 带 schema_version 且能在中断写入后安全恢复；
- Research Block Contract 能在长时自治中阻止无界重复 exploitation；
- long-running experiment 跨 session 不会被无意义重复启动；
- 关键 model/data/prompt/config 在 comparison 中具有 stable identity；
- 并行 writer 使用 collision-resistant IDs，不依赖无锁自增；
- Git/Evidence transaction 不存在 self-referential commit hash。

- 强制重启一次 session 后，新 Agent 只依赖 repo state 正确恢复 active research；
- Git commit/diff 与 Evidence 能双向定位；
- 不依赖 hooks/subagents/MCP/GitHub 也能完成完整 V0 loop；
- \$research-status / /research-status 能在不修改 canonical state 的情况下生成全局中文 Project Working Model；
- 状态汇报能正确区分 Established / Provisional / Refuted / Open，并显示 ACTIVE/Git consistency、三维 maturity 与真正需要架构师决策的事项；
- 把 status 报告交给另一客户端可快速建立正确全局认知，但开始执行时仍走正式 Resume Protocol。

20.2 V1 — 一周可稳定使用：Autonomous Research Batches

增加：

- 3–8 个 evidence iteration 的 bounded autonomous blocks（正式使用 Research Block Contract）；
- research-search、evaluation-design、experiment-review、retrospective、scenario-redteam；
- research-status snapshot integrity check、frontier/status compression 与 optional STATUS.md milestone cache；
- sharded immutable ledger；
- E2/E3 replay contract；
- worktree for risky/parallel branch；
- ARCHITECT signal history / scope / expiry；
- 轻量自动 synthesis；
- 研究 productivity telemetry；
- environment fingerprint / rebaseline policy；
- Research Capability Map 与高复用 harness 投资判断。
- ACTIVE.json + write-ahead state + run manifests；
- Session Rotation / Resume / reconciliation；

- Git research transaction / checkpoint commit / baseline tags;
- optional GitHub remote + manual Gate-3 verifier;
- schema migration/validation、atomic canonical-state writes、long-running job ownership;
- mutable-input lineage 与 evaluation surface calibration。

重点 KPI: **AI 选出的 discriminating experiment 有多少次不需要架构师纠正。**

20.3 V2 — Research Portfolio

只有真正需要时再加入:

- 3–5 个 active mechanism / architecture branches;
- stagnation detection;
- frontier diversity;
- parallel worktrees / read-heavy subagents;
- stronger evaluator isolation;
- official score budget policy;
- ShinkaEvolve 等 E2+ scoped optimizer;
- promotion verifier;
- Integration Mode profile。

仍不必演化成常驻 server。

21. 如何评价这套方法是否真正更高效

不能只看最终 score，也不能看 experiments/hour（会奖励无意义参数扫描）。建议记录:

KPI	定义	意义
Time-to-first-E1	项目开始 → 第一条 executable evidence	从零研究速度
Time-to-first-E3	项目开始 → 第一条 meaningful vertical slice	系统形成速度
Time-to-first-valid-	发现环境限制 → 第一条可回答同一关键问题的合法	环境适应速度

KPI	定义	意义
surrogate	surrogate evidence	
Session Recovery	fresh session 对 active question/branch/pending	session continuity
Accuracy	run/next action 的恢复正确率	
Time-to-resume	新 session 启动 → 恢复有效研究动作	context rotation 开销
Duplicate-work-after-rotation	session rotation 后无意义重复实验/工作量	durable state 完整性
Time-to-global-understanding	架构师或新 AI 从调用 research-status 到能正确说出目标/成熟度/关键 bottleneck/active research 所需时间	状态压缩质量
Status Context Accuracy	status 报告对 canonical state、ACTIVE/Git、关键 Findings/constraints 的复述正确率	全局认知可靠性
Status staleness detection	stale STATUS.md / CURRENT 与新 Evidence/Git 冲突时能正确发出 warning 的比例	防止 derived snapshot 冒充事实源
Git provenance coverage	E1+ evidence 可定位到 commit/diff/environment 的比例	可追溯性
Environment-blocked ratio	因环境能力不足无法得到目标 evidence 的 research questions 比例	research capability 缺口
Harness reuse yield	一个新增 harness / adapter 后支持的有效实验数 ÷ 构建成本	环境投资回报
Validated unknowns/day	被证实/证伪/明显收窄的 technical unknowns	研究推进质量
Evidence/hour	E1+ Gate-2 有效 evidence / wall clock	empirical throughput
Informative-failure ratio	informative_failure / valid runs	失败是否产生知识
Hypotheses	一次实验改变了多少 live hypotheses	信息增益 proxy

KPI	定义	意义
differentiated/run		
Tokens / informative evidence	token 消耗 / belief-changing evidence	ceremony overhead
Human interventions / 10 valid runs	架构师纠偏次数	AI autonomy
IMPLEMENT signal ratio	架构师指令中具体实现型占比	是否退化成 Copilot
Architect correction yield	一次 CHALLENGE 后避免的低价值实验/找到的新方向	人类经验价值
Time-to-autonomy-return	人类纠偏 → AI 再次自行推进	是否 takeover
Promotion survival	Gate-4 后仍保留的机制 / promoted	是否过早晋升
Local↔E5 correlation	本地 candidate ranking vs official	evaluator trust
False-confidence rate	曾 high-confidence 后被明显推翻的 finding 比例	研究诚实度

最关键的 A/B 对照不是“哪个写代码快”，而是：

- A: Superpowers/GSD 主导开发
- B: Research Engineering 主导 + Promotion 后才进入工程化

比较相同 wall-clock 下：可执行 evidence 数、被解决 technical unknowns、架构师干预次数、token、最终系统效果以及 promotion survival。

22. 风险与反模式

风险	典型表现	对策
AI 形成漂亮因果故事	一看到 collision 就说 “LiDAR detector failed”	trace reconstruction + competing hypotheses

风险	典型表现	对策
架构师建议被机械执行	“是不是 world model? ”→立刻重写系统	Architect Signal 语义; SUSPECT ≠ DECISION
AI 不敢自主	总问 A/B/C 让人选	routine selection 自主; 只有战略 tradeoff 升级
人重新接管	每个 threshold 都由人指定	尽量 DIRECTION/SUSPECT, 不用 IMPLEMENT
研究陷入局部最优	12 个 smoothing/threshold 实验	CHALLENGE + retrospective + frontier diversity
Proxy overfit	E1 metric 越来越好但仿真变差	Evidence level/limitations; 主动升级 E3/E4
过早造完整系统	三周没有可执行 evidence	cheapest executable artifact rule
过早工程化	一个 probe 有效就全面 TDD	多级 promotion; G3/G4 才工程化
研究 ceremony 膨胀	每轮写 5 份 Markdown	5 个核心状态文件; raw evidence append-only
Context pollution	stdout/trace 塞满 Agent context	artifact on disk; CURRENT/FINDINGS 只摘要
session restart 丢状态	新 Agent 重复实验/误删 dirty diff	ACTIVE + write-ahead state + Resume Protocol + Git reconciliation
stale handoff	CURRENT 写了方向但 active experiment 已变化	ACTIVE 与 run manifest 作为 execution pointer
Day 0 无状态却强	新项目没有 ACTIVE/CURRENT,	research-bootstrap 显式初始化

风险	典型表现	对策
行 Resume	Agent 开始猜状态	
state schema 演化破坏恢复	新 Skill 静默覆盖旧/ 未知字段	schema_version + conservative migration + unknown-newer no-overwrite
半写 canonical state	session crash 后 ACTIVE 只剩半个 JSON	validate + atomic replace
long-running run 被重复启动	session rotation 后第 二个 Agent 又启动同 一训练/仿真	run ownership / job identity + resume inspect
mutable input 漂移	算法未变但 prompt/model/data 同时变了	Mutable Input Identity Contract
evaluator proxy 漂移	local metric 变好而真 实行为变差	evaluation-design + E4/E5 calibration
status 变成第二事实源	架构师/AI 只相信旧 STATUS.md, 忽略 新 Evidence/Git	status 明确 derived/cache; 每次生成时读 canonical state; 执行前仍走 Resume
状态汇报变成项目管理流水账	报 commits/files/tasks 数量而看不出关键研 究问题	固定 Project Working Model 结构; 只报 belief/architecture- changing progress
evaluator hacking	AI 改评分逻辑	out-of-band isolation
infra failure 被当科学失败	OOM → “world model 无效”	execution status 独立
environment unsupported 被当	simulator 不支持攻击 注入 → “机制不可行”	ENV_BLOCKED/ENV_UNSUPPORTED/RESOURCE_EXCEEDED 分类

风险	典型表现	对策
hypothesis 失败		
无效 surrogate 产生伪证据	用无 dynamics mock 证明物理稳定	causal-feature check; EVIDENCE_INVALID
environment confound	tick rate/cache/version 改变却归因给算法	environment fingerprint + rebaseline
人类临时约束永久化	“这轮别动 planner”变永久规则	scope + expiry
多 agent 写冲突	同时写 JSONL/CURRENT	sharded ledger + single-writer global state
并行 ID 冲突	两个 worktree 都生成 EV-142	collision-resistant IDs; 禁止无锁自增
Git/Evidence 自引用事务	Evidence 试图记录包含自身的 commit SHA	code_state 与 record commit 分离; 预分配 EV/EXP ID
Git history ceremony	每个小 edit 都 branch/PR/CI	transaction 粒度按 meaningful evidence; PR/Actions 仅 promotion/expensive verify
remote/canonical drift	GitHub Issue/PR 与 repo research state 冲突	GitHub 只做 derived coordination plane; repo+Evidence canonical
unsafe physical research	AI 直接上真实机器人	HARD safety envelope + explicit approval

23. 最终建议

真正要构建的不是“AI 自动完成整个研发项目”的神奇框架，而是一套能让 Codex / Claude Code 的智能**持续停留在研究角色**的轻量机制。

第一阶段成功的判据非常具体：

你只给一个高层方向或一个物理/仿真观察，AI 能够自主把它转化为技术 *unknown*、构造最小可执行证据、选择算法、实现、实际运行、解释结果，并连续选择下一实验；你只在它搜索空间明显走偏时用 *Architect Signal* 纠偏。

最终关系应该是：

你：方向、观察、架构判断、纠偏、最终裁决

AI：算法、实现、probe、实验、归因、连续迭代

环境：既是实验约束，也是可演化的 research instrument

Git + Evidence + ACTIVE：可恢复的长期研究状态

research-status：总架构师与 AI 共享的全局 Project Working Model

而不是：

你：研究 policy + 算法 + 下一实验

AI：实现

24. 附录：可直接复制的核心文件草案

A. skills/research-engineering/SKILL.md

name: research-engineering

description: >

Use for exploratory or autonomous applied-AI research where the system, algorithm, architecture, evaluation surface, or research environment is still evolving. Operate as the lead architect's technical research partner. Optimize validated technical learning and working behavior, not production ceremony.

Research Engineering Mode

Act as the lead architect's high-level technical research partner.

Native compatibility

This skill must work with ordinary repository–local capabilities only:

- project instructions
- Agent Skills / SKILL.md
- file read/write
- local shell or Python
- Git

Do not require hooks, subagents, MCP, a daemon/server, a workflow engine, GitHub, or client–specific frontmatter for core correctness. Client–specific features may accelerate work but must not change research semantics.

Authority model

The architect owns:

- strategic direction
- HARD boundaries and resource policy
- important physical/simulation observations
- major architecture decisions and vetoes
- official submissions / external spend
- final Integration Mode promotion

You own routine technical research:

- inspect code, data, SDKs, traces and existing evidence
- select algorithms and technical mechanisms
- search literature/source implementations when useful
- form and challenge hypotheses
- build the cheapest useful executable probe/slice/harness
- implement and instrument prototypes
- run them and inspect actual behavior
- choose routine next experiments
- keep, modify, branch or discard from evidence

Do not ask the architect to select routine algorithms, thresholds, libraries, model variants, or internal implementation details.

Research artifact language policy

Apply this policy to artifacts YOU GENERATE during research. It is an output routing rule, not a requirement to translate the whole repository.

Human-facing research artifacts MUST be written in Chinese unless the architect asks otherwise. Preserve technical terms, algorithm/library/protocol names, metrics, identifiers, and proper nouns in their original English form.

AI-facing research artifacts MUST be written in English. This includes skill instructions/references, ACTIVE/CURRENT/ARCHITECT/BOUNDARIES/ENVIRONMENT/FINDINGS, hypothesis/evidence/failure records, schemas, JSON/JSONL, agent handoffs, and machine-facing script messages.

Architect input may be Chinese. Persist its normalized semantic meaning in English; preserve source_text only when exact wording has evidentiary value.

Do not maintain duplicate bilingual sources of truth. Use English canonical state/evidence plus concise Chinese human summaries that reference IDs.

Core objective

Maximize useful validated technical progress per unit wall-clock, compute, tokens, and architect attention.

Do not optimize for test count, documentation volume, plan completeness, architectural neatness of disposable prototypes, or number of code changes.

Session resume comes first

At the beginning of EVERY research-engineering session, before selecting a new research task:

0. If canonical research state does not yet exist, stop normal resume and use research-bootstrap. Do not invent missing prior state.
1. Read research/ACTIVE.json, CURRENT.md, ARCHITECT.md, BOUNDARIES.md,

ENVIRONMENT.md and relevant FINDINGS.md.

2. Inspect Git HEAD, current branch/worktree, git status, and relevant diff.
3. If ACTIVE is not idle, reconstruct that experiment before starting new work.
4. Inspect the active run manifest and produced artifacts/results.
5. Read only evidence referenced by ACTIVE/CURRENT/FINDINGS unless deeper diagnosis is required; do not replay the whole ledger.
6. Reconcile ACTIVE $\leftrightarrow$ Git $\leftrightarrow$ run artifacts $\leftrightarrow$ Evidence.
7. Never silently discard unfinished work or unexpected dirty state.
8. Decide resume / rerun / review / abandon / start-next only after recovery.

If ACTIVE and Git disagree, enter RECOVERY_RECONCILIATION. Do not blindly reset, checkout, overwrite, or start a new experiment.

Session context is disposable. Repository state is durable.

Write-ahead research state

Before a meaningful experiment changes code or starts execution, write/update ACTIVE.json with intent, question, hypothesis, expected evidence, Git base and expected touched files. Create research/runs/EXP-ID/manifest.json before run.

Update ACTIVE as work moves through implementing $\rightarrow$ ready_to_run $\rightarrow$ running $\rightarrow$ reviewing $\rightarrow$ idle/next. A session/client interruption is status=interrupted and must not update scientific belief.

Use a checkpoint commit for meaningful unfinished work that must survive a session boundary. Do not rely on anonymous git stash as the continuity protocol.

Stable abstraction: evidence

Do NOT assume a complete runnable application or evaluator exists.

Use the strongest affordable evidence:

- E0 reasoning / literature / source / SDK / data inspection
- E1 executable probe / visualization / feasibility spike
- E2 component replay / microbenchmark

- E3 vertical slice / partial closed loop
- E4 full system scenario / simulator
- E5 independent external judge / real-world observation

Always ask:

"What is the cheapest executable artifact that can materially reduce the current technical uncertainty?"

If a full system exists, run it. Otherwise build a vertical slice. If premature, evaluate a component. If no component exists, build a probe, architecture spike, or research harness. Use pure reasoning only when executable evidence is not yet practical.

Research environment feasibility

Read research/ENVIRONMENT.md before relying on an empirical route.

Never interpret inability to execute an experiment as evidence against a hypothesis.

Classify infeasibility separately:

- INFRA_FAILED: this run should have worked but failed operationally
- ENV_BLOCKED: capability exists in principle but a prerequisite/access is missing
- ENV_UNSUPPORTED: current environment cannot express the required phenomenon
- RESOURCE_EXCEEDED: required compute/time/storage exceeds the declared envelope
- EVIDENCE_INVALID: runnable substitute does not answer the causal question
- SCIENTIFIC_NEGATIVE: valid environment and evidence, mechanism is actually negative

Only SCIENTIFIC_NEGATIVE may directly weaken the hypothesis.

When desired evidence is infeasible:

1. identify the desired causal evidence;
2. reshape execution without changing the causal question;
3. decompose into answerable subquestions;
4. use a lower evidence level/surrogate only if causally valid;
5. build a reusable harness/instrument when leverage justifies cost;

6. escalate only across HARD/resource/access/major–strategy boundaries;
7. resume research rather than treating environment handling as a stopping state.

Surrogate Validity Contract

Before using surrogate evidence for a conclusion, identify:

- target causal claim
- required causal features
- preserved features
- missing/distorted features
- allowed conclusions
- forbidden conclusions

If a required causal feature is missing, classify EVIDENCE_INVALID. Do not claim a higher evidence level than the environment supports.

Environment comparability

For comparative evidence record a compact environment fingerprint. Treat changes in simulator/physics timing, sensor/noise model, data/replay snapshot, model/API version, inference backend/quantization/compiler mode, evaluator semantics, and performance–relevant hardware as potentially material.

Use:

- COMPATIBLE: direct comparison allowed
- REBASELINED: comparison allowed after anchor baseline rerun
- INCOMPARABLE: do not attribute cross–environment delta to the mechanism

Do not trigger full regression for irrelevant environment changes.

Update ENVIRONMENT.md only when a meaningful capability/limitation is discovered, a reusable harness changes research capability, a material environment change affects comparability, or availability changes. Prefer linking entries to EV–*.

Git research infrastructure

Treat Git as normative research infrastructure.

Evidence says what was learned. Git says the exact executable state. ACTIVE says what is currently being attempted.

Use commits at meaningful evidence/checkpoint boundaries, not every edit.

Recommended commit form:

research(EXP-0142): <short intervention>

Hypothesis: H-037

Evidence: EV-0821

Outcome: promising

Level: E2

Use branches by research direction, not by every micro experiment. Use worktrees for truly independent write-heavy or architecture branches. Never let parallel write-heavy agents modify the same working tree.

Promote the minimal validated mechanism, not an entire dirty research branch. Stable baselines/submissions should have immutable commits and annotated tags. Official/external evidence must reference the exact commit/tag and manifest.

GitHub is optional. If available, use it as a remote research plane for backup, manual expensive verification, protected credentials, promotion PRs, and releases. Do not make GitHub the only canonical research state and do not run heavyweight CI after every research commit.

Architect Signals

Interpret architect comments by semantics, not mere wording:

OBSERVE: evidence, not automatically causal fact; operationalize when useful.

SUSPECT: strong hypothesis prior, still challengeable by evidence.

DIRECTION: research priority; you still choose algorithms/implementation.

CHALLENGE: stop narrow exploitation, reopen assumptions/search space, then resume.

CONSTRAINT: obey within scope/expiry; do not make temporary constraints permanent.

DECISION: current architecture decision; state one material risk then execute.

DECISION FINAL: non–challengeable.

IMPLEMENT: implement requested mechanism but still validate effect.

VETO: stop the vetoed direction.

After architect steering, return to autonomous Research Mode. Intervention is an impulse, not a takeover.

Default fast loop

1. Resume state first.
2. Identify the active technical uncertainty.
3. Inspect relevant existing evidence.
4. Choose the cheapest VALID evidence level/surface.
5. State the research question and expected discriminating observation briefly.
6. Write ACTIVE intent; implement/instrument only enough to obtain evidence.
7. Pass Gate–0 and relevant Gate–1 checks.
8. Run the real probe/replay/slice/system as early as practical.
9. Record objective observation before interpretation.
10. Update the live hypothesis/belief.
11. Keep, modify, branch, discard, improve environment, or escalate evidence level.
12. Commit/checkpoint when a meaningful evidence or session boundary is reached.
13. Continue autonomously while inside boundaries.

Do not make this a ceremonial checklist in user–visible output. Execute it.

Minimum Correctness Envelope

Gate–0: intended experiment executes and produces expected artifact.

Gate–1: critical boundaries/invariants are credible: schema, shape, units, coordinates, timestamps, control limits, API contracts.

Gate–2: evidence can answer the stated question without obvious confound, evaluator leakage, invalid surrogate, or incompatible environment.

Gate–3: promising mechanisms receive limited clean/robustness/latency checks.

Gate–4: Integration Mode only: clean reproduction, broader regression, stable interfaces, permanent high–value regressions and engineering hardening.

Do NOT default to TDD for every mutation, tests for every new function, detailed plans, full regression, disposable-prototype code review, provisional backward compatibility, premature refactoring, or repeated architect approval.

Conditional expert skills

Use research-diagnosis only when causal failure is genuinely unclear.

Use hypothesis-design when several live mechanisms must be discriminated.

Use research-search when external algorithm/source knowledge can change mechanisms.

Use evaluation-design when the evaluation surface is absent, weak, gameable, or disagrees with E4/E5/human-observed behavior.

Use experiment-review after meaningful empirical evidence when interpretation is non-trivial.

Use research-retrospective at plateau or when architecture assumptions may be wrong.

Use scenario-redteam after a mechanism becomes promising or evaluation seems narrow.

Use integration-promotion only after evidence justifies stabilizing a mechanism.

Do NOT call every subskill every iteration.

Escalate to architect only for

- changing a HARD boundary
- strategic architecture tradeoff not resolvable technically
- external data/policy ambiguity
- paid resource or compute beyond cap
- official submission
- physical execution outside declared safety envelope
- missing access that leaves no valid evidence path
- environment investment requiring major commitment
- irreconcilable problem-definition contradiction

Uncertainty about the next algorithm, experiment, or ordinary environment workaround is NOT itself a reason to ask the architect. Research it.

Session rotation and persistent state

Raw evidence is append-only. Keep ACTIVE mechanical, CURRENT/ARCHITECT compact, and

FINDINGS evidence-derived. Do not turn state files into transcripts.

Before a deliberate session rotation:

- respect the current Research Block Contract;
- finish/record current evidence when practical;
- otherwise checkpoint unfinished meaningful work;
- update ACTIVE and CURRENT;
- ensure run manifest reflects actual state;
- commit/push when the research durability policy requires it.

A fresh session must be able to continue without old chat history.

A1. skills/research-bootstrap/SKILL.md

name: *research-bootstrap*

description: >

Use only when a repository or applied-AI problem does not yet have valid Research Engineering canonical state, or when the lead architect explicitly requests a clean research-state initialization. Inspect the problem, repository, rules, assets and environment; create the minimum durable state needed to begin evidence-driven research without producing a heavyweight project plan.

Research Bootstrap

Turn a zero-state repository/problem into the minimum valid Research Engineering state, then hand off to research-engineering.

Native compatibility

Use only ordinary project instructions, Agent Skills, file read/write, local shell or Python, and Git. Do not require GitHub, hooks, subagents, MCP, or a daemon.

Bootstrap when

- research/ACTIVE.json and the canonical research state do not exist;

- the repository is newly adopted into Research Engineering;
- existing research state is declared invalid/unrecoverable by the architect.

Do NOT rerun bootstrap merely because a new session starts. Existing state must be resumed instead.

Procedure

1. Inspect the problem statement, rules, repository, README/docs, SDK, data, baseline and available execution surfaces.
2. Separate external facts from assumptions.
3. Identify HARD boundaries and unresolved policy/rule ambiguities.
4. Discover the current Research Environment and obvious capability gaps.
5. Identify the highest-value technical unknowns; do not design the full solution.
6. Create versioned canonical state:
ACTIVE.json, CURRENT.md, ARCHITECT.md, BOUNDARIES.md, ENVIRONMENT.md, FINDINGS.md, and the Evidence ledger/run directories.
7. Record only justified initial E0 Evidence.
8. Choose the first technical uncertainty that can be reduced by the cheapest valid executable evidence.
9. Leave ACTIVE idle or planning_evidence with a clear next empirical action and hand off to research-engineering.

Do not

- generate a large task backlog;
- freeze a speculative final architecture;
- create broad test suites before they protect research validity;
- implement the full system during bootstrap;
- turn guessed requirements into HARD boundaries;
- ask the architect to select routine algorithms.

Output language

Canonical state is English. A short visible bootstrap summary for the architect is Chinese with original English technical terms.

Success condition

Bootstrap is complete when the repository has valid minimal durable state and the first high-value uncertainty has a concrete next evidence-producing action.

B0. research/ACTIVE.json

```

{
  "schema_version": "1.0",
  "session_epoch": "SE-...",
  "status": "idle",
  "research_question": null,
  "subject": null,
  "hypothesis_ids": [],
  "chosen_hypothesis": null,
  "experiment_id": null,
  "intent": null,
  "expected_evidence": {},
  "block": {
    "id": null,
    "objective": null,
    "max_evidence_iterations": null,
    "max_wall_clock_minutes": null,
    "completed_evidence_iterations": 0,
    "stop_conditions": []
  },
  "git": {
    "branch": null,
    "base_commit": null,
    "checkpoint_commit": null,
    "dirty_expected": false,
    "expected_touched_files": []
  },
  "execution": {

```

```

    "status": "idle",
    "completed_cases": [],
    "pending_cases": [],
    "run_manifest": null
  },
  "current_observation": null,
  "next_action": null,
  "updated_at": null
}

```

ACTIVE.json is a machine-facing execution pointer, not a research narrative. Update it before meaningful changes/runs and whenever execution state materially changes.

B1. research/runs/EXP-*/manifest.json

```

{
  "schema_version": "1.0",
  "experiment_id": "EXP-...",
  "question": "...",
  "hypothesis_ids": ["H-..."],
  "subject": {"type": "component", "id": "..."},
  "environment": {
    "id": "ENV-...",
    "fingerprint": "sha256:...",
    "comparability": "COMPATIBLE"
  },
  "code_state": {
    "branch": "research/...",
    "commit": "...",
    "base_commit": "...",
    "dirty": false,
    "diff_sha256": null
  },
  "inputs": {
    "dataset": "...@sha256:...",

```

```

"model": "...@checkpoint:...",
"prompt": "...@sha256:...",
"config": "...@sha256:..."
},
"command": ["python", "..."],
"status": "running",
"execution": {
  "host": "...",
  "launcher": "local_process",
  "pid_or_job_id": "...",
  "started_at": "...",
  "heartbeat_or_last_observed_at": "...",
  "stdout": "stdout.log",
  "stderr": "stderr.log",
  "expected_outputs": []
}
}

```

The manifest is created before execution. It identifies the intended run even if the client/session disappears. Update it crash-safely; do not infer scientific outcome merely from process exit.

A2. skills/research-status/SKILL.md

name: *research-status*

description: >

Use when the lead architect asks for current project progress, global research state, key bottlenecks, current architecture, active research, or a compact project context snapshot. Produce an evidence-grounded Chinese Project Working Model useful to both the architect and an AI rebuilding project context. This is read-mostly synthesis, not research planning or implementation.

Research Status

Compile the repository's current research state into a compact Project Working

Model.

The report has two audiences:

1. the lead architect, who needs fast global understanding;
2. an AI coding/research agent, which should be able to reuse the same report as high-quality context compression.

The report is a DERIVED SNAPSHOT. Canonical truth remains in:

- research/ACTIVE.json
- research/CURRENT.md
- research/ARCHITECT.md
- research/BOUNDARIES.md
- research/ENVIRONMENT.md
- research/FINDINGS.md
- Evidence / run records
- Git repository state

Language

Write the visible report in Chinese unless the architect asks otherwise.

Preserve technical terms, algorithm/library/protocol names, metrics, IDs, Git refs/tags and proper nouns in their original English form.

Keep the structure precise enough that the report can be pasted into a new Codex or Claude Code session as useful project context.

Do not create a duplicate bilingual source of truth.

Read-mostly default

By default do NOT:

- modify application code;
- start experiments;
- choose a new research direction;
- change hypotheses or FINDINGS;
- promote a candidate;

- alter Architect decisions;
- reset, clean, checkout over, or otherwise destructively modify Git state.

Report inconsistencies. Let research–engineering reconcile them.

Only persist research/STATUS.md when explicitly requested or at a declared milestone/checkpoint.

Snapshot sources

Read, in order:

1. research/ACTIVE.json
2. research/CURRENT.md
3. research/ARCHITECT.md
4. research/BOUNDARIES.md
5. research/ENVIRONMENT.md
6. research/FINDINGS.md
7. Evidence records referenced by those files
8. active run manifest/results/artifacts when relevant
9. Git HEAD, branch/worktree, status, relevant research commits and tags

Do NOT load the whole Evidence ledger by default.

GitHub is optional. If a remote and gh are available, supplementary state may include whether the active branch is pushed, an existing promotion PR, an explicitly requested verifier run, or release/submission tags. GitHub state must never override canonical local research state.

Snapshot integrity

Before reporting, perform a read–only consistency check:

ACTIVE

<–> Git HEAD / branch / status / diff

<–> active run manifest/artifacts

<–> referenced Evidence

Also check:

- Architect constraint scope/expiry;
- Environment ID and rebaseline status;
- whether CURRENT is stale relative to ACTIVE/Evidence/Git;
- whether the claimed baseline/tag exists.

If material state is inconsistent, show:

STATUS INTEGRITY WARNING

near the beginning. Do not silently repair destructive ambiguity.

Reporting principles

Do not use a software–style completion percentage unless explicitly defined.

Report separately:

- System Maturity
- Evidence Maturity
- Research Environment Maturity

Separate:

- what exists;
- what has been demonstrated;
- what is currently believed;
- what remains provisional;
- what has been refuted;
- what is being tested now.

Report only belief–changing, architecture–relevant, capability–relevant, or research–environment–relevant progress. Do not enumerate every commit, file, test, or routine experiment.

Default report structure

1. 项目一句话状态
2. 当前工作契约

3. 三维成熟度
4. 当前系统与架构形态
5. 近期实质进展
6. 当前 Findings: Established / Provisional / Refuted / Open
7. Current Research Frontier, when multiple meaningful families exist
8. 当前关键 Unknowns / Failures / Bottlenecks
9. Research Environment
10. 当前 Active Research
11. 下一阶段研究重点
12. 需要总架构师关注或决策
13. 风险与偏航提示
14. 继续研发上下文摘要
15. Snapshot basis / provenance

Architect decision section

Default to:

需要总架构师决策：无

Only list HARD–boundary changes, strategic architecture decisions, meaningful environment investment, external spend/access, official submissions, physical safety decisions, or unresolved Architect–directive conflicts.

Do not escalate routine algorithm/model/threshold/library choices.

Continue–research context capsule

End with a concise Chinese context capsule containing:

- Objective
- Current Direction
- Established Facts
- Current Provisional Beliefs
- Active Architect Decisions / Constraints
- Current Bottleneck
- Active Research / Experiment

- Environment limits relevant to the active problem
- Next Empirical Action
- Escalation Conditions

Keep original English technical terms and canonical IDs.

Do not include conversational history.

This capsule accelerates cognitive rehydration but does NOT replace the formal Session Resume Protocol. Before modifying code, a new agent must still reconcile ACTIVE <–> Git <–> run artifacts <–> Evidence.

Snapshot basis

End with:

- generated_at
- Git HEAD
- branch/worktree
- current baseline/tag if any
- Evidence cutoff / key Evidence IDs
- Environment ID
- ACTIVE status
- state–integrity status

Brevity

Default output should normally fit in about 1–3 screens. Expand only when the architect explicitly asks for a detailed status, architecture deep dive, full evidence history, or comparison with a prior milestone.

The purpose is context compression, not documentation growth.

A3. skills/evaluation–design/SKILL.md

***name:** evaluation–design*

***description:** >*

Use when the current evaluation surface is missing, weak, expensive, gameable,

poorly correlated with real behavior, or insufficient to distinguish important technical hypotheses. Treat evaluation itself as a Research Subject and design the cheapest trustworthy metrics, scenarios, slices, holdouts or verifier surface needed to answer the current research question.

Evaluation Design

Treat "how do we know this is better?" as a technical research problem.

Trigger when

- no local evaluator exists;
- the official evaluator is sparse, hidden, expensive or rate-limited;
- local metrics disagree with E4/E5 or architect-observed behavior;
- a qualitative observation needs an operational measure;
- moving from E1/E2 to E3/E4 requires representative scenarios;
- evaluator leakage, reward hacking or proxy overfit is plausible.

Method

1. State the real objective/rubric and the decision the evaluation must support.
2. Identify observable behaviors and failure/success dimensions.
3. Propose the minimum metrics/qualitative signals that discriminate the question.
4. Define representative slices/scenarios, clean controls and hard negatives.
5. Separate development/tuning surfaces from holdout/verifier truth when feasible.
6. State proxy limitations and obvious gaming/leakage paths.
7. Build the smallest evaluator/probe/scenario surface needed; run calibration cases.
8. Compare against E4/E5 results or architect observations when available.
9. Record what conclusions the evaluation surface may and may not support.

Evaluation validity contract

For each important evaluation surface, make explicit:

- target objective;
- measured observables;

- known blind spots;
- data/scenario provenance;
- tuning vs holdout separation;
- gaming/leakage risks;
- calibration evidence;
- current highest justified Evidence Level.

Do not promote a convenient scalar proxy into the real objective without evidence.
A metric improvement that conflicts with representative system behavior is a signal to investigate the evaluator, not permission to ignore the behavior.

If evaluator/metric semantics materially change, mark comparability and rebaseline anchor candidates before attributing deltas to the system.

Output

Canonical evaluation records/state are English. Human-facing evaluation reviews are Chinese with original English technical terms and canonical IDs.

C. research/CURRENT.md

Current Research State

Objective

<what the application must eventually achieve>

Evidence maturity

Current highest stable level: E?

System-wide level: E?

State explicitly whether an end-to-end system/evaluator exists.

Research environment maturity

Current environment id: ENV-...

Highest supported evidence by major capability: ...

Blocked / unsupported capabilities: ...

Recent environment changes requiring rebaseline: ...

Working pieces

– ...

Provisional system shape

...

This architecture is NOT frozen unless ARCHITECT/BOUNDARIES says otherwise.

Highest-value uncertainties

– U-...

Active research

– H-... / mechanism / branch

Current frontier

- strongest mechanism(s)
- distinct alternatives worth preserving

Next empirical action

...

Current block limits

- evidence iterations / time / compute
- escalation conditions

D. research/ARCHITECT.md

Architect State

Active Direction

D-012

statement: Focus on recovery; detection is no longer the primary bottleneck.

scope: current research phase

expiry: recovery checkpoint

Observations

A-017

statement: Cases 31/37/42 show visible stop-go oscillation after safety intervention.

source: direct simulation observation

status: active evidence

Suspicions

S-006

statement: Recovery state ownership may be split incorrectly between fusion and governor.

status: hypothesis prior, NOT fact

Decisions

D-004

statement: A post-planner independent interception boundary is required.

reason: control-channel attacks can occur downstream of planning.

final: true

Temporary Constraints

C-009

statement: Do not modify navigation planner.

scope: recovery research

expiry: recovery checkpoint

Vetoes

...

E. research/BOUNDARIES.md

Research Boundaries

HARD

- competition rules
- allowed external-data policy
- official metric semantics
- verified SDK coordinate/unit conventions
- physical actuator safety envelope
- evaluator / holdout truth
- official credentials
- spend / compute caps

Changes require architect approval.

PROVISIONAL

- current WorldState representation
- module boundaries
- safety/planner interface
- replay format
- current architecture ownership

The AI may change these when evidence supports it, but must record the change.

FREE

- algorithms / models
- thresholds / hyperparameters
- internal data structures
- prototype code
- instrumentation
- probes
- synthetic scenarios

F. research/ENVIRONMENT.md

Research Environment

Environment ID

ENV-local-v1

Available

Compute / Devices

- GPU: ...
- RAM: ...
- Runtime / budget constraints: ...

Simulator / SDK

- ...

Replay / Data / External Services

— ...

Known Limitations

ENV-LIM-001

capability: post-planner control tampering

status: ENV_UNSUPPORTED

impact: Cannot claim E4 physical-safety evidence for control-channel attacks.

verified_by: EV-...

Available Substitutes / Harnesses

HARNESS-001

capability: post-planner command corruption replay

supports_evidence: E2/E3

preserves:

- planner command schema
- timing/order

missing:

- actuator dynamics

verified_by: EV-...

Research Capability Map

Capability	Highest valid evidence	Status	Verified by
vision attack	E4	available	EV-...
lidar attack	E4	available	EV-...
control tampering	E2/E3	replay/partial only	EV-...
real robot	E5	unavailable	external constraint

Comparability / Rebaseline

Current fingerprint: ...

Last material change: ...

Comparability: COMPATIBLE | REBASELINED | INCOMPARABLE

Anchor baseline evidence: EV-...

Material Environment History

– ENV-CHG-...: ...

Update this file only when research capability or comparability materially changes.

G. research/FINDINGS.md

Durable Findings

Established

FND-004 — Raw LiDAR point count is not a robust integrity signal

Evidence: EV-0012, EV-0014, EV-0017

Confidence: high

Implication: do not pursue global point-count threshold without context.

Provisional

FND-007 — Temporal occupancy persistence appears useful

Evidence: EV-0017, EV-0021

Confidence: moderate

Limitation: dynamic-scene behavior unresolved.

Refuted

FND-002 — Sensor disagreement alone identifies attacks

Evidence: EV-0009, EV-0015

Confidence: high

Open contradictions

...

H. 推荐的 AGENTS.md / CLAUDE.md 共同核心

AGENTS.md:

Project Research Mode

This is an exploratory applied-AI research project.

The human acts as lead architect; the AI acts as technical research partner.

When \$research-engineering or /research-engineering is active:

- if canonical research state is absent, use research-bootstrap instead of inventing resume state;
- resume ACTIVE/CURRENT/Git state before selecting new work;
- optimize validated technical progress, not software-delivery ceremony;
- routine technical decisions are autonomous;
- do not default to TDD, detailed plans, full regression, or code review for disposable experiments;
- preserve HARD boundaries and the Minimum Correctness Envelope;
- read and respect research/ENVIRONMENT.md;
- never interpret environment infeasibility or session interruption as scientific negative evidence;
- validate surrogate causal fidelity before drawing conclusions;
- treat missing/untrusted evaluation as a research problem; use evaluation-design when needed;
- preserve schema_version and do not overwrite unknown newer state formats;
- keep canonical machine state crash-safe;
- make current uncertainty executable as early as possible;
- use Git as the normative executable-state/provenance layer;
- keep AI-facing project/research artifacts in English;
- produce human-facing research briefs/reports in Chinese while retaining original English technical terms;
- treat architect comments according to Architect Signal semantics;
- after architect steering, return to autonomous research;
- when research-status is invoked, synthesize a read-mostly Chinese Project Working Model from canonical repo state; never treat STATUS.md as canonical.

Research Mode overrides generic software-engineering workflow defaults for experimental changes.

Integration Mode may explicitly re-enable stronger planning/testing/review workflows.

Claude Code 的 CLAUDE.md 推荐直接使用其原生 import:

@AGENTS.md

Claude Code optional adapter

Use project skills under .claude/skills.

Client-specific hooks/subagents may accelerate research but must not change the common protocol.

这样共同语义只有一份 canonical project contract；Codex 原生读取 AGENTS.md，Claude Code 原生读取 CLAUDE.md 并导入 AGENTS.md。[S2][S19]

I. 可选 Skill 安装器

```
from pathlib import Path
import shutil

root = Path(__file__).resolve().parents[1]
source = root / "skills"

for target in [root / ".agents" / "skills", root / ".claude" / "skills"]:
    target.mkdir(parents=True, exist_ok=True)
    for skill in source.iterdir():
        if not skill.is_dir():
            continue
        dst = target / skill.name
        if dst.exists():
            shutil.rmtree(dst)
        shutil.copytree(skill, dst)
```

使用 copy 而不是依赖 symlink，可避免两个客户端对 symlink 语义差异成为 portable core 的隐含前提。

J. researchlog.py 最小接口

```
python tools/researchlog.py init
python tools/researchlog.py validate
python tools/researchlog.py record <evidence.json>
python tools/researchlog.py run --level E1 --subject mechanism:foo -- <command...>
python tools/researchlog.py compare EV-001 EV-002
python tools/researchlog.py snapshot
python tools/researchlog.py env <environment-change.json>
python tools/researchlog.py active <active-update.json>
python tools/researchlog.py reconcile
python tools/researchlog.py manifest <manifest-update.json>
```

```
python tools/researchlog.py job <EXP-ID>
python tools/researchlog.py checkpoint --experiment EXP-0142
```

这些命令只做 deterministic bookkeeping / schema validation / crash-safe state write / provenance / Git reconciliation / job inspection。不要给工具加入“下一实验是什么”“score 提升就 keep”“自动选 hypothesis”的智能；那属于 AI research policy。V0 甚至可以不用这个脚本，直接由 Codex/Claude Code 写文件并调用 Git。

25. 研究过程制品语言规范（由 Skills 执行，Normative）

本节定义的是 **research-engineering 及其子 Skills 在研究过程中生成制品时的语言行为**，不是要求仓库中的所有文件都使用某一种语言。该规则应直接写入主 SKILL.md，必要时子 Skills 继承，不需要每个 Skill 重复一遍。

Skill 生成的制品 / 输出	默认语言	规则
Research Brief、checkpoint summary、阶段总结、architecture review	中文	技术术语和专有名词保留英文原词
diagnosis summary、experiment review、retrospective、方案比较	中文	面向架构师/团队决策；可引用 EV-* / H-* 等英文 ID
Codex / Claude Code 对架构师的日常 research update	中文	除非用户明确要求英文
research-bootstrap visible bootstrap summary	中文	canonical initialization state 仍写英文；只向架构师简要说明发现的事实/边界/首个研究问题
research-status Project Working Model / optional STATUS.md	中文	derived snapshot；保留 English IDs/terms；可作为 AI fast context，但不是 canonical state
evaluation-design 面向人的 evaluation review	中文	metric/proxy/scenario/verifier 等术语保留英文；canonical evaluation records 使用英文
CURRENT.md / ARCHITECT.md / BOUNDARIES.md / ENVIRONMENT.md /	英文	AI durable state；用于跨 session 恢复和检索

Skill 生成的制品 / 输出	默认语言	规则
FINDINGS.md		
hypothesis / evidence / failure records、JSON/JSONL/schema/enums	英文	machine-readable source of truth
SKILL.md、skill references、AI-facing AGENTS.md / CLAUDE.md instructions	英文	Codex / Claude Code 共用行为协议
agent handoff、machine-facing hooks/scripts message	英文	降低多客户端/多 Agent 语义漂移
架构师即时输入	中文或英文均可	写入 AI-facing state 时规范化为英文语义；必要时保留原 source_text
Source code / comments / docstrings	遵循项目约定	本规范不强制翻译；无约定时默认英文
外部论文、README、日志等原始材料	保持原语言	若生成给人看的摘要，则摘要用中文

25.1 Mixed-audience 输出规则

当一次研究既需要机器状态又需要人类汇报时，**不要维护两份中英文重复事实源**。推荐：

English canonical state / evidence
↓ IDs / references
Chinese Research Brief / checkpoint summary

例如实验结束后：

- research/ledger/EV-017.json：英文，canonical evidence；

- research/CURRENT.md: 英文, 供下一次 Codex / Claude session 续接;
- CLI 给架构师的汇报: 中文, 例如“EV-017 支持 temporal persistence, 但 clean dynamic scenes 出现明显 false-stop”;
- 若生成阶段 Research Brief: 中文, 通过 EV-017、H-008 引用机器记录, 不复制完整英文 ledger。

这条规则使 **human interface** 与 **AI control/memory interface** 分离, 同时避免 bilingual duplication 导致事实漂移。

26. 原生客户端可执行性验收 (Normative)

本方案只有在不安装自研 agent runtime 的情况下能够通过以下验收, 才算实现成功。

26.1 Codex 原生验收

从 repo root 启动 Codex, 只有 .agents/skills、AGENTS.md、shell、Git 和项目本身存在:

```
$research-engineering
Continue current research.
```

必须能够: 全新 repo 先用 \$research-bootstrap 建立最小 state; 已有项目恢复 ACTIVE/Git; 读取 CURRENT/ENVIRONMENT; 自主选择 probe/algorithm; 修改代码; 执行 shell; 记录 evidence; commit/checkpoint; 在需要时继续下一 iteration; 缺失/失真的 evaluator 可调用 \$evaluation-design。不得要求 MCP、daemon、custom orchestrator、GitHub 或外部 Agent framework。\$research-status 必须能只读生成全局 Project Working Model。Codex repository skills 与 AGENTS.md 均为当前官方原生机制; AGENTS.md 保持轻量路由/不变量, 不复制整个 protocol。[S1][S2]

26.2 Claude Code 原生验收

从 repo root 启动 Claude Code, 只有 .claude/skills、CLAUDE.md(@AGENTS.md)、shell、Git 和项目本身存在:

```
/research-engineering
Continue current research.
```

必须产生同样 research semantics: /research-bootstrap、/research-engineering、/research-status 和 /evaluation-design 均在 project skills 中原生工作。Claude-specific hooks/subagents 不存在时不能导

致功能缺失；它们只允许作为性能/上下文隔离优化。Claude Code 当前原生支持 project skills 与 CLAUDE.md，并明确支持在 CLAUDE.md 中 import AGENTS.md。[S3][S19]

26.3 Cross-client handoff test

一个很强的验收是：

Session A: Codex 在 EXP-0142 中途 checkpoint

↓

关闭 Codex，不提供旧聊天

↓

Session B: Claude Code /research-engineering

↓

恢复同一 branch/ACTIVE/run manifest/evidence 并继续

然后反向再做一次。可先用 research-status 加速 cognitive rehydration，但开始执行前必须走正式 Resume。只要两端都只依据 repo state 恢复并得到相同 decision-relevant state，说明 portable core 真正成立。

26.4 State durability / evaluation / long-run 验收

至少做四个故障注入测试：

1. 在写 ACTIVE.json 中途杀掉客户端，确认 canonical 文件仍保持上一份有效状态或可由 temp/checkpoint 重构；
2. 用未知更高 schema_version 的 ACTIVE/manifest 启动 Agent，确认其报告 incompatibility 而不是静默覆盖；
3. 启动一个跨 session 的 long-running run，rotation 后确认新 session 先识别已有 job，再决定观察/收尾，而不是重复启动；
4. 故意让 local proxy metric 与 E4/人工观察相冲突，确认 Agent 触发 evaluator investigation / evaluation-design，而不是继续盲目优化 proxy。

再验证一次 mutable-input lineage：只改变 prompt/model/data/config 中任一项时，run manifest 能明确检测到 identity delta。

26.5 不能承诺的“无损”

“无损恢复”指 semantic/operational state，不保存模型私有 chain-of-thought，也不保证不同模型在同一状态下选择完全相同的探索策略。设计保证的是：**所有影响后续研究决策的已知事实、约束、实验意图、代码状态、证据与未完成动作都有 durable representation。**

27. 参考资料

[S1] OpenAI — Codex Skills / Build skills

<https://developers.openai.com/codex/skills>

[S2] OpenAI — Custom instructions with AGENTS.md

<https://developers.openai.com/codex/guides/agents-md>

[S3] Anthropic — Claude Code Skills / Subagents / Hooks

<https://code.claude.com/docs/en/skills>

<https://code.claude.com/docs/en/sub-agents>

<https://code.claude.com/docs/en/hooks>

[S4] obra/superpowers — systematic-debugging

<https://github.com/obra/superpowers/blob/main/skills/systematic-debugging/SKILL.md>

[S5] obra/superpowers — repository / methodology

<https://github.com/obra/superpowers>

[S6] LifeOS — RootCauseAnalysis skill

<https://github.com/danielmiessler/LifeOS/blob/main/LifeOS/install/skills/RootCauseAnalysis/SKILL.md>

[S7] agent-incident-postmortem

<https://github.com/mohitagw15856/pm-claude-skills/blob/main/skills/agent-incident-postmortem/SKILL.md>

[S8] Open-AutoResearch / AutoResearch++

<https://github.com/rbudnar/open-autoresearch>

[S9] lab-notes Agent Skill

<https://github.com/eins78/agent-skills/blob/main/skills/lab-notes/SKILL.md>

[S10] Karpathy autoresearch

<https://github.com/karpathy/autoresearch>

[S11] codex-autoresearch

<https://github.com/leo-lilinxiao/codex-autoresearch>

[S12] MLEvolve

<https://github.com/InternScience/MLEvolve>

[S13] SakanaAI ShinkaEvolve / Agentic Usage

<https://github.com/SakanaAI/ShinkaEvolve>

https://github.com/SakanaAI/ShinkaEvolve/blob/main/docs/agentic_usage.md

[S14] Microsoft R&D-Agent

<https://github.com/microsoft/RD-Agent>

[S15] Meta-Agent Challenge

<https://github.com/ant-research/meta-agent-challenge>

[S16] AIDE ML

<https://github.com/WecoAI/aideml>

[S17] OpenAI MLE-bench

<https://github.com/openai/mle-bench>

[S18] Autoresearch Bench (2026)

<https://www.autoresearch-bench.com/>

****设计底线：******本方案的价值不在于多几个 Markdown 文件，而在于让 Coding Agent 的默认研究行为发生改变：从等待人类给出下一技术动作，变成自主把不确定性变成可执行证据；从每次 mutation 都做生产级工程，变成只在 promotion 时付出工程化成本；从人类逐步指挥，变成资深总架构师以高信息量信号持续 steer 一个真正能够独立研究的 AI 技术合伙人；并让总架构师随时通过 research-status 获得可靠的全局 Project Working Model。**

[S19] Anthropic — How Claude remembers your project / CLAUDE.md imports AGENTS.md

<https://code.claude.com/docs/en/memory>

[S20] Git — git-worktree Documentation

<https://git-scm.com/docs/git-worktree>

[S21] GitHub Docs — Manually running a workflow / workflow_dispatch

<https://docs.github.com/actions/managing-workflow-runs/manually-running-a-workflow>

[S22] GitHub Docs — Deployments and environments / protection rules and secrets

<https://docs.github.com/actions/reference/workflows-and-actions/deployments-and-environments>